



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Pereira da Silva

**Arquitetura por Etapas e Estrutura Extensível para Refatoração Automática de Código
de Teste**

Florianópolis
30 de agosto de 2026

SUMÁRIO

1	INTRODUÇÃO	4
1.1	PROBLEMA DE PESQUISA	7
1.2	HIPÓTESE DE PESQUISA	8
1.3	PROPOSTA	8
1.4	OBJETIVOS	8
1.5	ORGANIZAÇÃO DO TRABALHO	8
2	FUNDAMENTAÇÃO TEÓRICA	9
2.1	TESTE DE SOFTWARE	9
2.1.1	Categorias e estratégias de configuração de acessórios	11
2.1.1.1	<i>Estratégia configuração local</i>	<i>11</i>
2.1.1.2	<i>Estratégia configuração implícita</i>	<i>12</i>
2.1.1.3	<i>Estratégia configuração delegada</i>	<i>13</i>
2.1.1.4	<i>Categoria de estratégias de configuração de acessórios compartilhados</i>	<i>14</i>
2.1.2	Test smells	14
2.2	MÉTRICAS DE SIMILARIDADE	15
2.2.1	Métricas de similaridade de código	15
2.2.1.1	<i>Métricas baseadas em medidas do código</i>	<i>16</i>
2.2.1.2	<i>Métricas baseadas em texto</i>	<i>17</i>
2.2.1.3	<i>Métricas baseadas em tokens</i>	<i>17</i>
2.2.1.4	<i>Métricas baseadas em árvores</i>	<i>18</i>
2.2.1.5	<i>Métricas baseadas em grafo</i>	<i>18</i>
2.3	ALGORITMOS DE AGRUPAMENTO	19
2.3.1	Métodos hierárquicos	19
2.3.2	Métodos particionais	19
2.3.3	Métodos baseados em densidade	20
2.3.4	Métodos baseados em modelo	20
2.3.5	Métodos baseados em grade	20
2.3.6	Métodos de computação suave	21
3	ESTADO DA ARTE	22
3.1	REVISÃO EXPLORATÓRIA	22
3.1.1	Reuso de código de teste	22
3.1.2	Teste de software e métricas de similaridade	24
3.2	ESTUDO SISTEMÁTICO	25
3.2.1	Discussão	30
4	PROPOSTA	32

4.1	ARQUITETURA EM <i>PIPELINE</i>	32
4.2	<i>FRAMEWORK</i> RÓŽA	34
4.2.1	Análise sintática	35
4.2.2	Medição	37
4.2.2.1	<i>JPlag</i>	37
4.2.2.2	<i>Simian</i>	37
4.2.2.3	<i>Deckard</i>	38
4.2.2.4	<i>LCS</i>	38
4.2.2.5	<i>LCCSS</i>	39
4.2.2.6	<i>Comparação das métricas</i>	41
4.2.3	Etapas ainda não implementadas	41
5	AVALIAÇÃO	42
5.1	OBJETIVO E DESENHO DO EXPERIMENTO	42
5.2	FONTE DE DADOS	44
5.3	MATRIZ DE CONTROLE	44
5.4	VARIANTES DE CONFIGURAÇÃO	44
5.5	EXECUÇÃO	45
5.6	MÉTRICAS DE AVALIAÇÃO E RESULTADOS	45
5.7	DISCUSSÃO DOS RESULTADOS	48
5.8	AMEAÇAS À VALIDADE	48
6	CONCLUSÃO	50
6.1	PRÓXIMOS PASSOS	50
	REFERÊNCIAS	52
	APÊNDICE A – VIOLAÇÕES DE CÓDIGO DE TESTE REGISTRADAS NA ANÁLISE SINTÁTICA	58
A.1	VIOLAÇÕES DE CLASSE	58
A.1.1	Importação com curinga	58
A.1.2	Mais de uma classe de nível superior no mesmo arquivo	58
A.1.3	Classe aninhada	59
A.1.4	Record aninhado	59
A.1.5	Herança	60
A.1.6	Classe abstrata	60
A.1.7	Classe genérica	60
A.1.8	Anotação na classe	61
A.1.9	Inicializador de classe	61
A.1.10	Construtor explícito	62

A.1.11	Enumerador	62
A.1.12	Atributo estático	62
A.1.13	Atributo anotado	63
A.1.14	Atributo inicializado no campo	63
A.1.15	Método auxiliar	63
A.1.16	Método de finalização ou demais métodos de ciclo de vida	64
A.1.17	Mais de um método de configuração implícita	64
A.1.18	Método de configuração implícita fora do contrato	64
A.2	VIOLAÇÕES DE MÉTODO	64
A.2.1	Teste parametrizado ou com ciclo de vida distinto	65
A.2.2	Anotação repetida no mesmo método	65
A.2.3	Anotação distinta de teste ou de configuração implícita	65
A.2.4	Atributo nas anotações de teste ou de configuração implícita	66
A.2.5	Método de teste fora do contrato	66
A.2.6	Laço	66
A.2.7	Comando try	67
A.2.8	Comandos switch, break, continue ou throw explícito	67
A.2.9	Bloco sincronizado ou comando rotulado	67
A.2.10	Expressão lambda ou referência a método	68
A.2.11	Classe anônima ou classe local	68
A.2.12	Record local	68
A.2.13	Expressão explícita this ou super	69

1 INTRODUÇÃO

Teste de software é uma das atividades presentes no processo de desenvolvimento e manutenção de software. De acordo com a versão 4.0 do *Guide to the Software Engineering Body of Knowledge* (SWEBOK) (Washizaki, 2024), teste de software consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Por muito tempo, teste de software foi visto sobretudo como uma etapa de verificação da qualidade do software produzido, e não como meio de construir um produto com qualidade (Murugesan, 1994). Nesse sentido, testes eram frequentemente realizados apenas ao final da etapa de codificação (Yang et al., 2011), com o objetivo de detectar falhas e fornecer um indicativo de que o software funciona conforme o esperado (Bertolino, 2007). Com o passar do tempo, a percepção sobre a utilidade do teste evoluiu. A atividade deixou de ser vista como etapa posterior ao desenvolvimento e passou a estar presente durante todo o processo de desenvolvimento e manutenção de software. Meszaros (2007) considera a atividade de teste como parte intrínseca do processo de desenvolvimento de software. Assim, a atividade de teste de software passou a incorporar propósitos variados, como detectar falhas (Tiwari; Goel, 2013), prevenir que erros sejam introduzidos (Beizer, 1990), prover um indicativo de que o software funciona (Bertolino, 2007), auxiliar na compreensão do projeto e do código do sistema (Freeman; Pryce, 2009) e servir como meio de especificação (Alvestad, 2007) e de documentação (Haugset; Hanssen, 2008) dos requisitos.

Nesse contexto, o código de teste deixou de ser um artefato auxiliar e passou a exigir os mesmos cuidados aplicados ao código de produção. Assim como o código da aplicação, o código de teste precisa ser mantido, compreendido e ajustado durante todo o projeto (Greiler; Deursen; Storey, 2013b). Para Meszaros (2007), a atividade de teste não deveria aumentar o esforço total de desenvolvimento e manutenção do software, uma vez que o esforço gasto com essa atividade deve ser compensado pela melhoria da qualidade do software. Entretanto, manter o código de teste apresenta desafios, principalmente quando a codificação e a evolução dos testes são realizadas diretamente pelo desenvolvedor. Conforme o software evolui e a quantidade de código aumenta, torna-se mais difícil manter os testes com consistência e sem perder o controle sobre eles. Um fenômeno semelhante ocorre no código de produção: sem refatoração contínua, cada nova funcionalidade tende a exigir mais esforço do que a anterior (Fowler, 1999). Com os testes, observa-se comportamento análogo. Novos testes passam a ser mais difíceis de adicionar e a duplicação tende a aumentar (Berner; Weber; Keller, 2005). Isso pode levar à degradação da manutenibilidade do código de teste (Greiler; Deursen; Storey, 2013a). Segundo Emery (2009), um motivo comum para o abandono de testes automatizados é o fato de os testes se tornarem frágeis e de alto custo de manutenção, o que pode tornar os desenvolvedores reticentes tanto em adicionar novos testes quanto em executar os existentes. Mesmo em um cenário em que tanto o código de produção quanto o código de teste são gerados por modelos de Inteligência Artificial (IA), a manutenibilidade do código de teste é um problema que precisa

ser tratado. Shamshiri et al. (2018) mostra que testes gerados automaticamente afetam o tempo e a confiança do desenvolvedor, e que tarefas de manutenção, envolvendo tanto a correção do código de produção quanto a atualização do código de teste, levaram 29% mais tempo quando realizadas com o apoio de testes gerados automaticamente. Nesse sentido, independentemente do contexto, a manutenibilidade do código de teste ainda é um aspecto importante no processo de desenvolvimento de software.

Evitar a duplicação no código de teste é uma das formas de melhorar a manutenibilidade. Isso ocorre porque mudanças na aplicação passam a ter impacto menor nos testes, e o desenvolvedor dispõe de menos código para manter e gerenciar (Berner; Weber; Keller, 2005; Greiler; Deursen; Storey, 2013a). De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam alto potencial de reuso; conforme o software evolui e o número de testes aumenta, é comum que a quantidade de código duplicado entre os testes também aumente. As Figuras 1 e 2 ilustram duplicação no código de teste. Cada figura apresenta um método de teste em uma classe distinta. Os exemplos utilizam a linguagem Kotlin¹ e o *framework* JUnit 5². O código das linhas 3–8 em ambos os testes configura o sistema em teste (*System Under Test* – SUT) da mesma forma, o que produz código de configuração duplicado.

```

1 @Test
2 fun 'deve cancelar a matrícula do estudante'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.matricular(alice)
9
10    val sucesso = algoritmos.cancelarMatricula(alice)
11
12    assertTrue(sucesso)
13    assertTrue(algoritmos.estudantes.isEmpty())
14 }

```

Figura 1 – Método de teste com configuração do SUT no próprio método, suscetível à duplicação de código

Fonte: Elaborado pelo autor.

Técnicas que reduzam essa duplicação podem contribuir para preservar a manutenibilidade do código de teste ao longo do projeto, bem como para diminuir o esforço total de desenvolvimento dos testes. Uma dessas técnicas é a configuração implícita (ou *implicit setup*), em que os testes são organizados em classes de teste, de forma que o código de configuração comum possa ser centralizado em um método de configuração (Meszaros, 2007). A Figura 3 ilustra a aplicação da configuração implícita. Além da remoção da duplicação de código, esse exemplo mostra como a utilização da configuração implícita traz mais clareza à compreensão do teste. O contexto comum (ter um aluno matriculado em uma disciplina) aparece apenas uma vez e cada método de teste foca no aspecto mais importante para aquele teste.

¹ <https://kotlinlang.org/>

² <https://junit.org/>

```

1 @Test
2 fun 'não deve matricular o estudante duas vezes'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.matricular(alice)
9
10    val sucesso = algoritmos.matricular(alice)
11
12    assertFalse(sucesso)
13    assertEquals(listOf(alice), algoritmos.estudantes)
14 }

```

Figura 2 – Método de teste com lógica de configuração do SUT idêntica à da Figura 1, ilustrando a duplicação

Fonte: Elaborado pelo autor.

```

1 class TesteUniversidade {
2     private lateinit var algoritmos: Disciplina
3     private lateinit var alice: Estudante
4
5     @BeforeEach
6     fun configurar() {
7         val ufsc = Universidade("UFSC")
8         alice = ufsc.admitir("Alice")
9         val bob = ufsc.contratar("Bob")
10        val cc = ufsc.criarPrograma("Ciência da Computação")
11        algoritmos = cc.criarDisciplina("Algoritmos", bob)
12        algoritmos.matricular(alice)
13    }
14
15    @Test
16    fun 'deve cancelar a matrícula do estudante'() {
17        val sucesso = algoritmos.cancelarMatricula(alice)
18
19        assertTrue(sucesso)
20        assertTrue(algoritmos.estudantes.isEmpty())
21    }
22
23    @Test
24    fun 'não deve matricular o estudante duas vezes'() {
25        val sucesso = algoritmos.matricular(alice)
26
27        assertFalse(sucesso)
28        assertEquals(listOf(alice), algoritmos.estudantes)
29    }
30 }

```

Figura 3 – Suíte de testes refatorada com configuração implícita, centralizando a lógica comum de inicialização e eliminando a duplicação

Fonte: Elaborado pelo autor.

Entretanto, nem sempre é simples decidir como os testes devem ser organizados. Por exemplo, considere o teste da Figura 4. Esse teste requer um estado do SUT diferente do estado comum dos testes da Figura 3. Em vez de uma disciplina com um aluno matriculado, esse teste requer uma disciplina fechada e sem alunos matriculados. Como nem toda configuração pode ser compartilhada, o novo teste não poderia ser colocado na mesma classe de teste dos demais

sem que o código da classe existente fosse adaptado. A pergunta que fica é: compensaria fazer essa refatoração? Por um lado, o código que é comum ao novo teste e aos demais (linhas 3–7 da Figura 4) deixaria de ser duplicado. Por outro lado, o código que realiza a matrícula do aluno na disciplina (linha 12 da Figura 3) precisaria ser duplicado nos dois testes que a exigem. Considerando apenas o reuso de código, é tácito concluir que compensaria colocar o novo teste na mesma classe de teste dos demais, uma vez que o reuso obtido seria maior que a duplicação introduzida; para reusar cinco linhas de código, uma linha de código precisaria ser duplicada duas vezes. Mas, e se ao invés de apenas dois, a classe existente já tivesse seis testes? Nesse caso, mover o novo teste para a mesma classe dos demais testes introduziria mais linhas duplicadas do que aquelas que seriam reusadas.

```

1 @Test
2 fun 'não deve matricular estudante em disciplina fechada'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.fechar()
9
10    val sucesso = algoritmos.matricular(alice)
11
12    assertFalse(sucesso)
13    assertTrue(algoritmos.estudantes.isEmpty())
14 }

```

Figura 4 – Método de teste que requer um estado distinto do SUT, impedindo o reuso da configuração comum da Figura 3

Fonte: Elaborado pelo autor.

O problema fica evidente mesmo com um exemplo simples: organizar os métodos de teste levando em conta a redução da duplicação de código não é uma tarefa trivial. Em especial, em projetos com centenas ou milhares de testes, a distribuição de um novo teste leva a uma explosão combinatória de possibilidades. Convém destacar que não é apenas uma decisão sobre em qual classe de teste inserir o novo teste, mas também sobre como redistribuir os testes já existentes entre as classes de teste de modo a reduzir a duplicação. Quando um novo teste é introduzido, por exemplo, uma distribuição que reduza mais a duplicação pode exigir a separação de testes que antes pertenciam à mesma classe de teste. Percebe-se, então, que é necessário refatorar e reorganizar continuamente o código de teste em busca de distribuições que reduzam a duplicação de código.

1.1 PROBLEMA DE PESQUISA

Diante desse contexto, o problema de pesquisa é enunciado da seguinte maneira: como reduzir a duplicação de código de teste sem aumentar o esforço total de desenvolvimento e manutenção e sem alterar o resultado dos testes?

1.2 HIPÓTESE DE PESQUISA

Considerando o problema apresentado, a hipótese de pesquisa deste trabalho é enunciada da seguinte forma: é possível reduzir a duplicação de código de teste e o esforço de desenvolvimento por meio de um método de refatoração automática que trate o reuso de código de teste como um problema global de redistribuição de métodos entre classes de teste.

1.3 PROPOSTA

Em linha com a hipótese apresentada, este trabalho propõe a utilização de métricas de similaridade e algoritmos de agrupamento para abordar o problema global de redistribuição de métodos entre classes de teste. As métricas de similaridade permitem identificar testes com código de configuração em comum; os algoritmos de agrupamento, por sua vez, propõem grupos de testes cuja redistribuição favorece o reuso de código pela estratégia de configuração implícita.

1.4 OBJETIVOS

O objetivo geral deste trabalho consiste em reduzir a duplicação de código de teste e o esforço total de desenvolvimento e manutenção dos testes, sem alterar o resultado dos testes. Para alcançar o objetivo geral, os seguintes objetivos específicos deverão ser alcançados: (1) investigar métricas de similaridade para identificar código de configuração em comum entre testes; (2) investigar algoritmos de agrupamento para propor distribuições de métodos entre classes de teste; (3) propor uma arquitetura em *pipeline* para refatoração de código de teste; (4) especificar e implementar um *framework* que instancia essa arquitetura sem alterar o comportamento observado dos testes; e (5) avaliar a abordagem quanto à redução de duplicação e ao esforço de refatoração.

1.5 ORGANIZAÇÃO DO TRABALHO

Os demais capítulos deste trabalho são organizados da seguinte forma: no Capítulo 2 é apresentada a fundamentação teórica; no Capítulo 3 é apresentado o estado da arte e os trabalhos relacionados; no Capítulo 4 é apresentada a proposta deste trabalho; no Capítulo 5 é apresentada a avaliação empírica das métricas de similaridade; e no Capítulo 6 é apresentada a conclusão e os próximos passos para elaboração deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Três tópicos se destacam em nível de importância para este trabalho: teste de software, métricas de similaridade e algoritmos de agrupamento. Neste capítulo são apresentados os principais conceitos envolvendo a implementação de testes automatizados, as diferentes famílias de métricas de similaridade de código e os métodos de agrupamento utilizados para formar grupos de elementos similares.

2.1 TESTE DE SOFTWARE

Um teste é um procedimento, manual ou automático, que pode ser usado para verificar se um dado sistema em teste funciona de acordo com o comportamento esperado. O SUT representa a parte do sistema que está sendo testada. Cada teste deve realizar, inicialmente, uma série de passos para colocar o SUT no estado desejado para que o teste possa ser executado. Os elementos necessários para que o SUT seja colocado no estado desejado recebem o nome de *test fixtures*, ou simplesmente *fixtures*, ou acessórios de teste. A parte da lógica do teste em que os acessórios são criados é chamada de *fixture setup*, ou configuração de acessórios. Após executar a configuração de acessórios, o teste exercita o SUT através da execução da operação que está sendo testada, realiza uma série de verificações para garantir que o SUT funciona de acordo com o comportamento esperado e, por fim, limpa os acessórios.

Os acessórios distinguem-se pela forma como são reutilizados e pelo local em que são armazenados. Um acessório novo (*fresh fixture*) é construído exclusivamente para um teste e descartado ao final da execução. Um acessório coletivo (*shared fixture*) é construído uma única vez e reutilizado por vários testes (Meszaros, 2007). Quando o acessório permanece apenas em memória, ele é transiente; quando é gravado em um meio externo (e.g., um banco de dados), ele é persistente (Meszaros, 2007).

Os exemplos apresentados neste trabalho são baseados no *framework* de teste JUnit, disponível para as linguagens Java¹ e Kotlin. Os exemplos utilizam a linguagem Kotlin. A Figura 5 apresenta uma classe de teste com dois métodos de teste. A anotação `@Test` nas linhas 2 e 12 indica ao *framework* que os métodos anotados correspondem a testes. Essa abordagem segue a definição utilizada por Freeman e Pryce (2009), na qual, tipicamente, cada teste é separado em um método de teste. Além disso, cada teste pode ser dividido conceitualmente em quatro fases (Meszaros, 2007): configuração (*setup*), exercício, verificação e finalização (*tear down*).

Considerando o primeiro teste da Figura 5, as linhas 4 e 5 correspondem à etapa de configuração. Nessa etapa, o SUT deve ser colocado no estado desejado para o teste. É na configuração que os acessórios necessários para o teste são definidos. Nesse caso, a instância da classe `RepositorioDeUsuario` é um acessório necessário para o teste, e a conexão com o

¹ <https://www.java.com/>

```

1 class TesteRepositorioDeUsuario {
2     @Test
3     fun deveInserirUsuario() {
4         val repositorio = RepositorioDeUsuario()
5         repositorio.conectar()
6         val inserido = repositorio.inserir("Alice")
7         assertTrue(inserido)
8         repositorio.remover("Alice")
9         repositorio.desconectar()
10    }
11
12    @Test
13    fun deveRemoverUsuario() {
14        val repositorio = RepositorioDeUsuario()
15        repositorio.conectar()
16        repositorio.inserir("Alice")
17        val removido = repositorio.remover("Alice")
18        assertTrue(removido)
19        repositorio.desconectar()
20    }
21 }

```

Figura 5 – Classe de teste com dois métodos que exercitam o repositório de usuários, ilustrando as etapas de configuração, exercício, verificação e finalização

Fonte: Elaborado pelo autor.

banco de dados é estabelecida na linha 5. A etapa seguinte consiste em exercitar o SUT. É nessa etapa que é executado o que efetivamente será testado. Na Figura 5, a linha 6 corresponde à etapa de exercício do primeiro teste. Na etapa de verificação, o comportamento esperado do sistema é verificado a partir das saídas produzidas pelo SUT ou através de mecanismos que permitam obter o estado interno do SUT (e.g., consultas no banco de dados). No primeiro teste da Figura 5, a verificação ocorre na linha 7. A última etapa consiste em limpar os acessórios definidos na configuração, para evitar que futuras execuções do teste, ou que execuções futuras de outros testes, venham a ser afetadas. No primeiro teste da Figura 5, a finalização ocorre nas linhas 8 e 9, com a remoção do usuário inserido e o encerramento da conexão com o banco de dados.

De acordo com Meszaros (2007), existem estratégias de agrupamento para organizar os testes em classes. Na estratégia *testcase class per feature*, ou classe de teste por funcionalidade da aplicação, os testes que verificam uma mesma funcionalidade, ou um mesmo requisito, são reunidos em uma classe, independentemente da classe de produção exercitada. O objetivo é facilitar a leitura dos testes como especificação daquela funcionalidade. Na estratégia *testcase class per class*, ou classe de teste por classe da aplicação, cada classe de produção possui uma classe de teste correspondente, que concentra os testes dos seus métodos. Essa organização simplifica a correspondência entre o código de produção e o código de teste. Entretanto, os testes da mesma classe podem exigir acessórios distintos entre si. Na estratégia *testcase class per fixture*, ou classe de teste por acessório, os testes são agrupados pelo estado inicial de que necessitam. Assim, testes que utilizam acessórios semelhantes ficam na mesma classe, o que permite centralizar a configuração comum, em especial pela estratégia de configuração

implícita.

O manifesto da automação de testes (Meszaros; Smith; Andrea, 2003) apresenta características esperadas dos testes automatizados. Os testes devem ser autoverificáveis (i.e., cada teste reporta o próprio resultado); repetíveis, de modo que possam ser executados múltiplas vezes; robustos, produzindo resultados consistentes para o mesmo código de aplicação; e independentes. O princípio de independência dos testes estabelece que cada teste deve poder ser executado isoladamente ou em conjunto com outros testes, em qualquer ordem, sem depender do resultado, do estado ou da ordem de execução dos demais (Meszaros; Smith; Andrea, 2003; Meszaros, 2007). Quando esse princípio é violado, o resultado de um teste passa a depender do estado deixado por outro, o que pode produzir falhas intermitentes e dificultar o diagnóstico. Inovações na atividade de teste devem preservar essas qualidades.

2.1.1 Categorias e estratégias de configuração de acessórios

A configuração de acessórios compreende a lógica do teste em que os acessórios são criados. Cada teste possui uma configuração de acessórios, que consiste na criação dos acessórios que serão utilizados no teste. A configuração de acessórios de um teste pode ser realizada através de diferentes estratégias, podendo ser uma ou mais estratégias ao mesmo tempo. Um dos problemas que afetam a manutenibilidade do código de teste é a duplicação do código de configuração de acessórios. Nem sempre é simples ou possível evitar esse tipo de problema, porém existem diferentes estratégias de configuração de acessórios que podem ser utilizadas para reduzir a duplicação de código.

Meszaros (2007) divide as estratégias de configuração de acessórios em duas categorias: *fresh fixture setup*, ou configuração de acessórios novos, e *shared fixture construction*, ou configuração de acessórios compartilhados. Em ambas as categorias existem estratégias que possibilitam o reuso de código. Entretanto, apenas na configuração de acessórios compartilhados é possível o reuso de execução. A categoria configuração de acessórios novos é composta por três estratégias: *inline setup*, ou configuração local; *implicit setup*, ou configuração implícita; e *delegated setup*, ou configuração delegada. Nessa categoria há apenas a possibilidade de reuso de código, mas não de reuso de execução. Já as estratégias da configuração de acessórios compartilhados realizam a criação de acessórios coletivos, os quais podem ser criados uma única vez e reutilizados entre várias execuções de teste distintas.

2.1.1.1 Estratégia configuração local

Na estratégia configuração local, os acessórios são criados dentro do próprio método de teste. Em geral, essa estratégia é utilizada em testes que necessitam de acessórios muito específicos ou no desenvolvimento dos primeiros testes, em que ainda não existe a necessidade de reuso de acessórios. A Figura 6 mostra um exemplo de teste em que a estratégia configuração local é utilizada. As linhas 4 e 5 contêm a configuração de acessórios do teste. Destaca-se que,

na estratégia configuração local, a configuração de acessórios é realizada diretamente no método de teste.

```

1 class TesteCriarBancoComConfiguracaoLocal {
2     @Test
3     fun criarBanco() {
4         val sistemaBancario = SistemaBancario()
5         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
6         assertEquals("Banco do Brasil", bb.obterNome())
7         assertEquals(Moeda.BRL, bb.obterMoeda())
8     }
9 }

```

Figura 6 – Método de teste com configuração local, na qual os acessórios são criados no próprio método

Fonte: Elaborado pelo autor.

Como os acessórios são criados diretamente no método de teste, torna-se fácil compreender a relação de causa e efeito entre os acessórios e as saídas do SUT. Entretanto, a longo prazo, essa estratégia tende a aumentar a duplicação de código, uma vez que diversos testes podem utilizar acessórios idênticos ou muito similares. Além disso, essa estratégia pode dificultar a compreensão do teste nos casos em que os acessórios forem muito complexos, uma vez que a quantidade de código aumenta e se torna mais difícil compreender o papel do acessório no teste.

2.1.1.2 Estratégia configuração implícita

Na estratégia configuração implícita, os acessórios são criados através de um método especial, comumente denominado de *setup method*, ou método de configuração, pertencente à classe de teste e executado antes de cada método de teste. A Figura 7 apresenta um exemplo da estratégia configuração implícita. Nesse exemplo, o método configurar da linha 5 representa o método de configuração. O atributo bb declarado na linha 2 é utilizado para permitir que o acessório seja manuseável pelos testes da classe. Quando um teste da classe é executado, o *framework* fica responsável por descobrir e executar o método de configuração. No JUnit, a anotação `@BeforeEach` é utilizada para indicar o método de configuração.

A vantagem dessa estratégia é promover o reuso de acessórios entre testes de uma classe. Porém, para utilizar essa estratégia, os testes que utilizam os mesmos acessórios devem ser agrupados na mesma classe. Isso pode trazer uma complicação a longo prazo, pois se torna cada vez mais difícil agrupar testes que necessitem exatamente dos mesmos acessórios. Com facilidade, alguns testes irão necessitar de acessórios específicos somente para eles. Além disso, nem sempre é possível evitar a duplicação de código através dessa estratégia, uma vez que um determinado teste pode ter acessórios similares a dois outros testes que, por sua vez, não possuam acessórios similares entre si. Greiler, Deursen e Storey (2013a) observa que essa estratégia pode fazer com que o método de configuração crie acessórios utilizados apenas por parte dos testes da classe.

```

1 class TesteCriarBancoComConfiguracaoImplicita {
2     private lateinit var bb: Banco
3
4     @BeforeEach
5     fun configurar() {
6         val sistemaBancario = SistemaBancario()
7         bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
8     }
9
10    @Test
11    fun criarBanco() {
12        assertEquals("Banco do Brasil", bb.obterNome())
13        assertEquals(Moeda.BRL, bb.obterMoeda())
14    }
15 }

```

Figura 7 – Classe de teste com configuração implícita, na qual os acessórios são criados em um método de configuração executado antes de cada teste

Fonte: Elaborado pelo autor.

2.1.1.3 Estratégia configuração delegada

A estratégia configuração delegada utiliza métodos auxiliares responsáveis pela criação dos acessórios. Em geral, esses métodos são disponibilizados através de classes auxiliares, separadas das classes de teste. A Figura 8 apresenta um exemplo da estratégia configuração delegada. O acessório necessário para o teste é criado através do método `criarBancoDoBrasil`, definido na linha 11. Esse método auxiliar é chamado na linha 4 da classe de teste.

```

1 class TesteCriarBancoComConfiguracaoDelegada {
2     @Test
3     fun criarBanco() {
4         val bb = Auxiliar.criarBancoDoBrasil()
5         assertEquals("Banco do Brasil", bb.obterNome())
6         assertEquals(Moeda.BRL, bb.obterMoeda())
7     }
8 }
9
10 object Auxiliar {
11     fun criarBancoDoBrasil(): Banco {
12         val sistemaBancario = SistemaBancario()
13         return sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
14     }
15 }

```

Figura 8 – Método de teste com configuração delegada, na qual a criação dos acessórios é delegada a um método auxiliar

Fonte: Elaborado pelo autor.

A principal vantagem dessa estratégia é permitir que o mesmo código de teste possa ser utilizado por testes diferentes de classes diferentes. A desvantagem dessa estratégia é que, em alguns casos, a sua utilização pode dificultar a compreensão da relação de causa e efeito entre os acessórios e as saídas do SUT, uma vez que o método auxiliar é, comumente, colocado em uma classe separada do teste.

2.1.1.4 Categoria de estratégias de configuração de acessórios compartilhados

Um exemplo descrito por Meszaros (2007) é a estratégia *lazy setup*, ou configuração preguiçosa, na qual o primeiro teste cria o acessório coletivo e os testes seguintes reutilizam a mesma instância, o que é útil quando a criação do acessório é custosa (e.g., a população de um banco de dados compartilhado entre os testes de uma suíte). Os *frameworks* de teste atuais não oferecem formas nativas e nem padronizadas para realizar esse tipo de configuração de acessórios. Por isso, é necessário que o desenvolvedor do teste assuma esse papel e passe a gerenciar parte do fluxo de execução dos testes.

2.1.2 Test smells

Fowler (1999) introduziu o termo *code smell* para caracterizar traços na estrutura interna do programa que indicam que a refatoração pode ser necessária. Um *code smell* não constitui defeito nem prova de erro funcional; é um indício de que a organização do código dificulta sua compreensão e evolução, e de que uma investigação mais detalhada pode revelar oportunidades de melhoria. No código de teste, os *test smells* estendem essa ideia: são sintomas de decisões de projeto que prejudicam a compreensão, a manutenibilidade e a eficácia da suíte (Deursen et al., 2001). Deursen et al. (2001) apresentaram o catálogo inicial; trabalhos posteriores ampliaram o conjunto de *test smells* e as técnicas de detecção (Greiler; Deursen; Storey, 2013a; Peruma et al., 2020). A seguir são definidos os *test smells* citados neste trabalho.

O *test smell* de acessórios generalizados (*general fixture*) ocorre quando o método de configuração cria acessórios que não são utilizados por todos os testes da classe (Deursen et al., 2001; Greiler; Deursen; Storey, 2013a). Com isso, testes que precisam de apenas parte da configuração passam a depender de um estado mais amplo do que o necessário, o que prejudica a legibilidade, e cada teste realiza trabalho desnecessário na criação de acessórios que não utiliza. O teste ansioso (*eager test*) ocorre quando um mesmo método de teste exercita diversos comportamentos do objeto de produção (Deursen et al., 2001). Com isso, o teste pode falhar por motivos distintos e torna-se mais difícil identificar o comportamento que ele verifica. A roleta de asserções (*assertion roulette*) ocorre quando um método de teste contém várias asserções sem uma mensagem que identifique qual delas falhou (Deursen et al., 2001). A asserção duplicada (*duplicate assert*) ocorre quando um mesmo método de teste verifica a mesma condição mais de uma vez (Peruma et al., 2020). Se a condição precisa ser exercitada com valores diferentes, cada caso deve constituir um método de teste próprio, cujo nome indique o caso verificado. Esse *test smell* surge, em geral, quando (1) vários casos são agrupados em um único método; (2) resta código de depuração; ou (3) há cópia acidental. Diferentemente da roleta de asserções, o problema não é a ausência de mensagem em verificações distintas, e sim a repetição da mesma condição no mesmo método. O teste de número mágico (*magic number test*) ocorre quando um método de teste utiliza um literal numérico como argumento de uma asserção, sem nomear o valor (Peruma et al., 2020). Com isso, o significado do número esperado fica implí-

cito, o que prejudica a leitura e a manutenção do teste. O emaranhado de asserções disjuntas (*Disjoint Assertion Tangle – DAT*) ocorre quando um mesmo método de teste verifica comportamentos logicamente independentes, que poderiam ser separados em testes distintos (Narang; Du; Jones, 2025). Com isso, uma falha mistura responsabilidades diferentes e dificulta localizar o comportamento afetado. A falta de coesão dos métodos de teste (*lack of cohesion of test methods*) ocorre quando métodos reunidos na mesma classe de teste não compartilham uma responsabilidade comum, cobrindo partes distintas do código de aplicação (Greiler; Deursen; Storey, 2013a).

A Figura 9 reúne os sete *test smells* em uma única classe. No método configurar (linhas 7–11), o acessório conta é criado e utilizado apenas por saldoDaConta, o que caracteriza o *test smell* de acessórios generalizados. O método nomeECriacaoDeConta (linhas 14–17) é um teste ansioso porque reúne a consulta do nome e a criação de uma conta, dois comportamentos distintos de Banco. O método dadosDoBanco (linhas 20–24) apresenta roleta de asserções: três verificações distintas sem mensagem que identifique qual delas falhou. O método nomeDoBanco (linhas 27–33) apresenta asserção duplicada: a mesma condição, o nome do banco, é verificada várias vezes com valores diferentes. O método moedaEQuantidadeDeBancos (linhas 36–39) apresenta DAT: a moeda do banco e a quantidade de bancos no sistema são comportamentos independentes reunidos no mesmo teste. O método saldoDaConta (linhas 42–45) apresenta o teste de número mágico: o literal 100.0 aparece na asserção sem uma constante que explique o valor. Por fim, a classe mistura testes de Banco, de Conta e de SistemaBancario, o que caracteriza a falta de coesão dos métodos de teste.

2.2 MÉTRICAS DE SIMILARIDADE

Métricas de similaridade possuem aplicações em diversas áreas da computação, como reconhecimento facial em imagens (Cao; Ying; Li, 2013), busca textual (Sahami; Heilman, 2006) e recuperação de informação estruturada (Dorneles et al., 2004). Métricas de similaridade determinam o grau de similaridade entre dois elementos, em que o grau de similaridade é um valor entre 0 e 1 que indica o quanto dois elementos são parecidos entre si de acordo com o critério definido pela métrica. Cada métrica de similaridade possui um mecanismo para gerar o grau de similaridade, de modo que cada métrica apresenta uma distribuição específica sobre o domínio de valores. Neste trabalho, pretende-se avaliar a similaridade entre testes através do código de teste. Por isso, métricas de similaridade de código são abordadas a seguir.

2.2.1 Métricas de similaridade de código

Avaliar a similaridade de código é uma atividade que possui propósitos variados, como identificação de fragmentos duplicados de código (Roy; Cordy, 2008), detecção de plágio e verificação de violação de direitos de cópia de software (Ottensstein, 1976), recomendação de código (Holmes; Murphy, 2005) e outros. Ragkhitwetsagul, Krinke e Clark (2018) classificam


```

1 class TesteSistemaBancario {
2     private lateinit var sistemaBancario: SistemaBancario
3     private lateinit var bb: Banco
4     private lateinit var conta: Conta
5
6     @BeforeEach
7     fun configurar() {
8         sistemaBancario = SistemaBancario()
9         bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
10        conta = bb.criarConta("Alice")
11    }
12
13    @Test
14    fun nomeECriacaoDeConta() {
15        assertEquals("Banco do Brasil", bb.obterNome(), "nome")
16        assertNotNull(bb.criarConta("Bob"), "conta")
17    }
18
19    @Test
20    fun dadosDoBanco() {
21        assertEquals("Banco do Brasil", bb.obterNome())
22        assertEquals(Moeda.BRL, bb.obterMoeda())
23        assertEquals(1, sistemaBancario.contarBancos())
24    }
25
26    @Test
27    fun nomeDoBanco() {
28        assertEquals("Banco do Brasil", bb.obterNome(), "Banco do Brasil")
29        val itau = sistemaBancario.criarBanco("Itau", Moeda.BRL)
30        assertEquals("Itau", itau.obterNome(), "Itau")
31        val santander = sistemaBancario.criarBanco("Santander", Moeda.BRL)
32        assertEquals("Santander", santander.obterNome(), "Santander")
33    }
34
35    @Test
36    fun moedaEQuantidadeDeBancos() {
37        assertEquals(Moeda.BRL, bb.obterMoeda(), "moeda")
38        assertEquals(1, sistemaBancario.contarBancos(), "quantidade")
39    }
40
41    @Test
42    fun saldoDaConta() {
43        conta.depositar(100.0)
44        assertEquals(100.0, conta.obterSaldo())
45    }
46 }

```

Figura 9 – Classe de teste com um exemplo de cada *test smell* definido nesta seção

Fonte: Elaborado pelo autor.

cinco diferentes abordagens de métricas de similaridade de código: baseadas em medidas do código, baseadas em texto, baseadas em *tokens*, baseadas em árvore e baseadas em grafo. Nas seções a seguir, essas cinco abordagens são apresentadas.

2.2.1.1 Métricas baseadas em medidas do código

Medidas do código como a quantidade de linhas e a quantidade de variáveis entre duas funções diferentes, podem ser usadas como indicativos de similaridade de código. As abordagens baseadas em medidas do código foram as primeiras relatadas na literatura e foram

aplicadas na detecção de similaridade de software. Ottenstein (1976) utiliza métricas de similaridade para identificar potenciais equivalências entre programas de alunos de uma disciplina de programação. A técnica contabiliza o número de operandos e operadores, com base na heurística de que a probabilidade de dois programas distintos apresentarem as mesmas contagens de operandos e operadores é baixa. Com isso, a métrica é robusta à renomeação de variáveis.

O problema de identificação de plágios entre trabalhos de disciplinas de programação também é abordado por Donaldson, Lancaster e Sposato (1981). Nessa abordagem, o número de declarações, chamadas, leituras, atribuições, comandos `while` e `if`, e definições de funções é contabilizado para avaliar a similaridade entre programas, permitindo a detecção de semelhanças estruturais.

2.2.1.2 Métricas baseadas em texto

Métricas de similaridade de código baseadas em texto avaliam a similaridade entre dois programas através da análise de sequências de caracteres do código-fonte. Roy e Cordy (2008) propõem a identificação de clones de fragmentos de código através de uma técnica que analisa diretamente o código-fonte. Diferente de abordagens que buscam identificar plágio, a proposta busca encontrar fragmentos de código intencionalmente copiados com o propósito de reuso. A abordagem representa o código como sequências de linhas e computa a similaridade utilizando o algoritmo da subsequência comum mais longa (*Longest Common Subsequence* – LCS) (Hirschberg, 1977). Antes de converter o código em sequências de linhas, a abordagem normaliza o código-fonte, removendo indentação e comentários e padronizando açúcar sintático.

Em outro trabalho, a similaridade entre dois programas é determinada através de uma matriz em que cada linha de um programa é comparada com todas as linhas do outro (Ducasse; Rieger; Demeyer, 1999). Quando duas linhas são idênticas, a entrada correspondente da matriz recebe o valor um; caso contrário, recebe o valor zero.

2.2.1.3 Métricas baseadas em tokens

Assim como as abordagens baseadas em texto, as abordagens baseadas em *tokens* também avaliam a similaridade entre dois programas através da análise textual do código-fonte. Abordagens baseadas em *tokens*, no entanto, aumentam o nível de abstração convertendo o código-fonte em sequências de *tokens*. A utilização de *tokens* visa normalizar diferenças textuais através da definição de tipos abstratos de constructos textuais (e.g., todos os identificadores ou valores literais presentes no código podem ser representados por um mesmo *token*).

Kamiya, Kusumoto e Inoue (2002) utilizam a abordagem baseada em *token* na ferramenta CCFinder. A ferramenta é capaz de identificar clones em diferentes linguagens, incluindo C, C++, Java e COBOL. Para cada linguagem suportada, um analisador léxico dedicado é utilizado para tokenizar o código-fonte. O método busca descobrir classes de clones (i.e.,

fragmentos de código que compartilham uma sequência idêntica de *tokens*). O resultado é a identificação e extração de rotinas comuns, incentivando o reuso de código.

2.2.1.4 Métricas baseadas em árvores

Diferente das categorias vistas anteriormente, esta categoria de técnicas tem o foco voltado para identificar a similaridade entre dois programas com base na semelhança estrutural entre eles. Nesse sentido, diferenças léxicas são menos relevantes na análise de similaridade. Para avaliar a similaridade entre dois programas são construídas árvores abstratas de sintaxe (*Abstract Syntax Tree* – AST). As ASTs permitem identificar a similaridade entre dois trechos mesmo com uma alta quantidade de modificações entre eles. A complexidade computacional, entretanto, é mais alta quando comparada com as categorias de técnicas vistas anteriormente.

Baxter et al. (1998) utilizam ASTs para identificar clones, ou clones que sofreram modificações, no contexto de programas escritos na linguagem C. O objetivo do trabalho é identificar clones criados intencionalmente, em que um desenvolvedor copia um fragmento existente para construir um novo fragmento semelhante. A detecção automática alerta os desenvolvedores para esses clones, de modo que a duplicação possa ser eliminada via refatoração. A partir das ASTs, a similaridade é computada dividindo o número de nós coincidentes pelo número total de nós. Um limiar de similaridade é então aplicado, e fragmentos cuja similaridade ultrapassa esse limiar são reportados como clones.

2.2.1.5 Métricas baseadas em grafo

Árvores podem ser vistas como uma especialização de grafos. Nesse sentido, métricas de similaridade baseadas em árvores se assemelham às métricas baseadas em grafos, uma vez que ambas permitem capturar aspectos estruturais do código-fonte. As métricas baseadas em grafo, no entanto, possuem como característica adicional a capacidade de identificar similaridade de aspectos semânticos. Essa capacidade adicional ocorre ao custo de maior complexidade computacional, inerente aos algoritmos sobre grafos.

Krinke (2001) utiliza grafos para representar a estrutura e o fluxo de dados de programas e, a partir disso, identificar duplicação de código. O objetivo é maximizar a identificação de trechos de código duplicados ainda que exista diferença entre os trechos, minimizando a ocorrência de falsos positivos (i.e., evitar a identificação de dois trechos de código como duplicados quando eles não são semanticamente equivalentes). Para detectar similaridade semântica, o autor utiliza grafos de dependência de programa (*Program Dependence Graph* – PDG) (Ferrante; Ottenstein; Warren, 1987), em que os nós correspondem a comandos no código e as arestas representam dependências de dados e de fluxo de controle. Komondoor e Horwitz (2001) também representam o código-fonte através de PDGs para identificar pares de clones. Ao aplicar algoritmos de detecção de subgrafos isomorfos, a abordagem pode encontrar clones não contíguos, clones com comandos permutados e até clones interdependentes.

2.3 ALGORITMOS DE AGRUPAMENTO

Agrupamento consiste na tarefa de atribuir padrões a grupos, ou *clusters*, de maneira não supervisionada (Jain; Murty; Flynn, 1999). Técnicas de agrupamento são empregadas para formar grupos de elementos mutuamente similares. O objetivo principal é maximizar a similaridade entre elementos do mesmo grupo (similaridade intragrupo) e minimizar a similaridade entre elementos pertencentes a grupos diferentes (similaridade intergrupo). Consequentemente, para uma coleção de entrada de elementos, uma técnica de agrupamento deve gerar, como saída, um conjunto de grupos, cada um contendo um ou mais elementos.

Há uma ampla variedade de métodos de agrupamento. Esses métodos podem ser organizados em seis categorias principais (Rokach; Maimon, 2005): hierárquicos, particionais, baseados em densidade, baseados em modelo, baseados em grade e métodos de computação suave. Cada categoria é caracterizada por um princípio de indução subjacente que conduz a operação dos algoritmos de agrupamento nela contidos.

2.3.1 Métodos hierárquicos

O agrupamento hierárquico forma grupos de maneira recursiva, seja fundindo sucessivamente agrupamentos em grupos maiores, seja dividindo-os em grupos menores. Um método hierárquico aglomerativo comumente citado inicia com n grupos, cada um contendo um único elemento (Johnson; Wichern, 2002). Em seguida, a cada passo, os dois grupos mais similares são combinados. Esse procedimento iterativo continua até que um critério de parada predefinido seja atingido.

A estrutura resultante dos grupos pode ser representada por um diagrama bidimensional em forma de árvore, denominado dendrograma (Sibbald; Wentzell; Wade, 1989). O dendrograma descreve a sequência de fusões realizadas em diferentes níveis da hierarquia. Em cada nível, os grupos podem ser fundidos utilizando uma de três estratégias de ligação: ligação simples, em que a similaridade se baseia no par de elementos mais próximos entre os grupos; ligação completa, que se apoia na maior distância entre elementos de grupos diferentes; e ligação média, que utiliza a distância média entre todos os pares de elementos dos dois grupos para quantificar a similaridade.

2.3.2 Métodos particionais

Métodos particionais, como K-means (Hartigan; Wong, 1979) e CLARANS (Ng; Han, 2002), partem de um conjunto predefinido de grupos e utilizam realocação iterativa para mover elementos de um grupo para outro, com o objetivo de atingir otimalidade global. No K-means, o número de grupos k é informado a priori. Cada grupo é representado por um centróide, calculado como a média dos elementos a ele associados, e cada elemento é atribuído ao centróide mais próximo. Os centróides são então atualizados e o processo de atribuição se repete até

que as associações se estabilizem. O CLARANS segue a mesma ideia de partição em k grupos, porém representa cada grupo por um medoide (i.e., um elemento efetivamente presente no conjunto) e explora o espaço de soluções por amostragem aleatória, o que o torna adequado a conjuntos volumosos de dados espaciais.

2.3.3 Métodos baseados em densidade

Métodos baseados em densidade consideram que os grupos correspondem a regiões densas do espaço de dados, separadas por regiões de baixa densidade. O algoritmo DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) (Ester et al., 1996) é um exemplo conhecido dessa categoria. Nesse algoritmo, um ponto é considerado central quando possui um número mínimo de vizinhos em um raio ϵ ; pontos alcançáveis a partir de um ponto central pertencem ao mesmo grupo, e pontos isolados são tratados como ruído. Diferentemente dos métodos particionais, o DBSCAN não exige que o número de grupos seja definido de antemão e é capaz de identificar grupos de formato arbitrário.

2.3.4 Métodos baseados em modelo

Métodos baseados em modelo partem de um modelo matemático para gerar hipóteses e, em seguida, tentam ajustar os dados de agrupamento a esse modelo. Além de descobrir grupos, esses métodos também podem fornecer descrições características para cada grupo. Em geral, são agrupados em dois tipos principais: métodos de árvore de decisão e métodos de redes neurais, como o mapa auto-organizável (*Self-Organizing Map* – SOM) (Kohonen, 2013). Nos métodos de árvore de decisão, o espaço de atributos é particionado de forma recursiva, de modo que cada grupo passa a ser descrito pelas condições que levam até o nó correspondente. No SOM, os elementos são projetados sobre uma grade de neurônios de baixa dimensionalidade; neurônios vizinhos ajustam seus vetores de peso em direção aos padrões apresentados, de forma que regiões contíguas do mapa passam a representar grupos de elementos similares.

2.3.5 Métodos baseados em grade

Métodos baseados em grade derivam grupos a partir de um conjunto finito de células dispostas em uma estrutura de grade. O algoritmo STING (*Statistical Information Grid*) (Wang; Yang; Muntz, 1997) é um exemplo dessa categoria. O espaço de dados é dividido em células retangulares em múltiplos níveis de resolução, e cada célula armazena estatísticas resumidas dos elementos que contém (e.g., contagem, média e extremos). O agrupamento opera sobre essas estatísticas, sem percorrer novamente cada elemento. O principal benefício desses métodos é a alta velocidade de processamento.

2.3.6 Métodos de computação suave

Métodos de computação suave (*soft computing*) incluem agrupamento *fuzzy*, recozimento simulado (*simulated annealing* – SA) (Kirkpatrick; Jr; Vecchi, 1983) e outras técnicas de otimização estocástica. No agrupamento *fuzzy*, um elemento pode pertencer a mais de um grupo simultaneamente, com graus de pertinência que quantificam a força dessa associação. O recozimento simulado trata o agrupamento como um problema de otimização: partições candidatas são perturbadas de forma aleatória e, no início do processo, soluções piores podem ser aceitas para escapar de mínimos locais; a probabilidade de aceitar uma solução pior diminui gradualmente, à medida que um parâmetro análogo à temperatura é reduzido.

3 ESTADO DA ARTE

Para o levantamento do estado da arte e a identificação de trabalhos correlatos foi realizada uma revisão exploratória da literatura, seguida de um estudo sistemático. A revisão exploratória teve como objetivo aprofundar a base de conhecimento sobre reuso de código de teste e sobre a interseção entre teste de software e métricas de similaridade. Já o estudo sistemático concentrou-se em trabalhos com objetivo semelhante ao deste trabalho (i.e., a refatoração automática ou semiautomática do código de teste). O objetivo principal da segunda etapa foi verificar a originalidade da proposta e o seu potencial de contribuição para o estado da arte, em especial no campo de teste de software.

3.1 REVISÃO EXPLORATÓRIA

O tema foi explorado a partir de trabalhos considerados empiricamente mais relevantes para esta pesquisa. A seleção utilizou as bases ACM Digital Library, Springer, IEEE Xplore e ScienceDirect. Além disso, a partir dos trabalhos selecionados, as referências consideradas mais relevantes também foram consultadas. A busca concentrou-se em técnicas de reuso de código de teste e métricas de similaridade aplicadas no contexto de teste de software. Convém destacar que, além da revisão exploratória, referências previamente conhecidas foram utilizadas na elaboração da fundamentação teórica.

3.1.1 Reuso de código de teste

A revisão exploratória identificou três trabalhos que propõem mecanismos de reuso de código de teste: DbUnit, Picon e Estória. Para situar essas abordagens em relação ao problema desta pesquisa, também são consideradas as estratégias de configuração implícita e de configuração delegada (Meszaros, 2007). A análise considera o escopo de reuso de cada técnica, a forma de identificar as oportunidades e o mecanismo empregado para centralizar o código compartilhado.

Christensen et al. (2006) apresentam o DbUnit, uma extensão do *framework* JUnit para reusar acessórios persistentes e coletivos em testes que envolvem tabelas de banco de dados. A aplicação da técnica exige identificar manualmente os testes que operam sobre as mesmas entidades e organizá-los em uma classe para cada tabela testada. As dependências entre essas classes devem reproduzir as dependências entre as tabelas, de modo que os testes das tabelas referenciadas por chaves estrangeiras sejam executados recursivamente antes dos testes da própria classe. Para manter o banco de dados em estado consistente, cada teste que insere um registro deve possuir um teste correspondente que o remova. O reuso restringe-se, portanto, ao código de inserção de entidades persistentes e acopla a execução dos testes à ordem das dependências do esquema relacional, o que pode violar o princípio de independência dos testes (Meszaros; Smith; Andrea, 2003).

Longo et al. (2015) propõem o Picon, que utiliza uma notação baseada em *JavaScript Object Notation – JSON*¹ para manter um repositório centralizado de acessórios, em especial objetos Java simples (*Plain Old Java Objects – POJOs*). Após identificar manualmente POJOs iguais utilizados por diferentes testes, o desenvolvedor descreve cada objeto uma única vez em um arquivo com a extensão *picon*. Um qualificador identifica o acessório, que se torna manuseável pela declaração de um atributo homônimo na classe de teste (e.g., o qualificador `programador:joao` corresponde ao atributo `programadorJoao`). Embora evite repetir o código de criação desses objetos, a abordagem exige uma linguagem específica distinta daquela em que os testes são escritos. Além disso, a injeção é dinâmica, de modo que inconsistências entre qualificadores e atributos somente são percebidas durante a execução.

Silva e Vilain (2017) apresentam a Estória, que promove o reuso de sequências iniciais da configuração de acessórios entre classes de teste. O desenvolvedor identifica manualmente as sequências compartilhadas e organiza as classes em uma hierarquia de dependências, na qual as classes mais abaixo reutilizam a configuração definida nas classes superiores. A abordagem admite tanto o reuso de código quanto o reuso de execução. No primeiro caso, a cadeia de configuração é executada novamente antes de cada teste, preservando a possibilidade de execução isolada e em qualquer ordem (Meszaros; Smith; Andrea, 2003). No segundo, a cadeia é reaproveitada entre testes de um mesmo ramo da hierarquia; a anotação `@Safe` identifica aqueles que não modificam os acessórios e permite reusar a execução nos testes seguintes. Esse modo reduz o tempo de execução, mas impõe uma ordem aos testes e viola sua independência. A identificação das sequências compartilhadas, a definição da hierarquia e a marcação dos testes seguros permanecem manuais.

Na configuração implícita (Meszaros, 2007), o reuso limita-se às sequências iniciais comuns aos testes de uma classe. O desenvolvedor identifica manualmente os testes com o mesmo prefixo de configuração, reúne-os em uma classe e desloca a sequência compartilhada para um método executado antes de cada teste. A estratégia mantém o código comum próximo aos testes, mas depende da organização das classes e não permite extrair trechos compartilhados que ocorram após uma divergência nas configurações.

Na configuração delegada (Meszaros, 2007), o reuso abrange trechos contíguos comuns localizados em diferentes posições do código de teste. Após identificar manualmente esses trechos, o desenvolvedor os extrai para métodos auxiliares e substitui cada ocorrência por uma chamada ao método correspondente. Como os métodos auxiliares podem ser chamados por testes de classes distintas, a estratégia não exige que todos os testes que compartilham o trecho sejam reunidos na mesma classe.

Em conjunto, as abordagens mostram que o ponto de reuso pode ser uma classe de teste, um método auxiliar ou um arquivo externo, e que cada escolha restringe as oportunidades de extração a um tipo de duplicação. DbUnit e Picon atuam sobre domínios específicos, respectivamente entidades persistentes e POJOs; Estória e configuração implícita concentram-se em

¹ <https://www.json.org/>

sequências iniciais; e configuração delegada alcança trechos contíguos em outras posições. Em todos os casos, porém, a identificação das oportunidades ou a organização necessária à refatoração depende da intervenção do desenvolvedor. Essa dependência limita a aplicação contínua das técnicas em suítes extensas e motiva o uso de métricas de similaridade e automação para localizar e refatorar o código compartilhado.

3.1.2 Teste de software e métricas de similaridade

Na revisão exploratória, dois tópicos se destacam na aplicação de métricas de similaridade ao teste de software: redução de suíte de teste (*test suite reduction*) e priorização de caso de teste (*test case prioritization*). A redução de suíte de teste (Harrold; Gupta; Soffa, 1993) seleciona, segundo um critério estabelecido, apenas um subconjunto dos testes de uma suíte, com o objetivo de reduzir o tempo total de execução. A priorização de caso de teste (Wong et al., 1997), por sua vez, preserva todos os testes, mas os ordena para antecipar a detecção de falhas. Em ambas as atividades, a similaridade pode ser usada para maximizar a diversidade do conjunto selecionado: quando dois casos de teste são semelhantes, um deles pode ser descartado na redução da suíte ou receber menor prioridade na priorização.

Ledru et al. (2012) utilizam métricas de distância entre as entradas dos casos de teste para priorizar aqueles menos semelhantes aos já selecionados. A abordagem compara as distâncias de Hamming (Hamming, 1950), Levenshtein (Levenshtein, 1966), cartesiana e Manhattan (Deza; Deza, 2009). Nos experimentos realizados, a priorização baseada em distância superou a ordenação aleatória quanto à média da porcentagem de falhas detectadas, e a distância Manhattan apresentou o melhor resultado médio. Entretanto, a representação considera apenas as entradas dos casos de teste e pode desconsiderar informações semânticas presentes no código.

Miranda et al. (2018) apresentam a família FAST, composta por abordagens de priorização baseadas em similaridade aplicáveis tanto a testes de caixa-preta quanto a testes de caixa-branca. Para testes de caixa-preta, o código de teste é transformado em conjuntos de caracteres, comparados pelo coeficiente de Jaccard (Jaccard, 1912); para testes de caixa-branca, utilizam-se informações de cobertura. Técnicas de busca aproximada reduzem o número de comparações entre os testes, o que permite aumentar a eficiência da priorização sem perda significativa de eficácia.

Três trabalhos investigam a redução de suítes produzidas por teste baseado em modelos (Hemmati et al., 2010; Hemmati; Briand, 2010; Hemmati; Arcuri; Briand, 2010). Os casos de teste abstratos são representados por caminhos em máquinas de estados, e algoritmos genéticos selecionam um subconjunto que maximiza a diversidade. Diferentes métricas são avaliadas, entre elas as distâncias de Hamming e Levenshtein, uma função de contagem e o coeficiente de Jaccard. Nos estudos industriais, o coeficiente de Jaccard aplicado aos conjuntos de gatilhos e guardas apresentou a melhor relação entre custo e taxa de detecção de falhas, e a seleção baseada em diversidade superou abordagens baseadas em cobertura ou seleção aleatória.

Além da redução e da priorização, Erfani, Keivanloo e Rilling (2013) aplicam detecção

de clones à recomendação de casos de teste. A abordagem associa testes existentes a fragmentos semelhantes do código de aplicação e recomenda esses testes para fragmentos correspondentes que ainda não possuem cobertura. Nesse caso, a similaridade é medida entre trechos do código de aplicação para localizar testes potencialmente reusáveis, os quais ainda precisam ser adaptados ao novo contexto. Assim, os trabalhos encontrados utilizam similaridade para selecionar, ordenar ou recomendar testes, mas não para identificar configurações de acessórios compartilhadas e refatorar a organização das classes de teste.

3.2 ESTUDO SISTEMÁTICO

Além da revisão exploratória, foi realizado um estudo sistemático para identificar trabalhos relacionados. A finalidade foi verificar se a proposta possui originalidade e potencial de contribuição para o estado da arte. Os trabalhos-alvo foram aqueles que envolvem a refatoração automática ou semiautomática de casos de teste, considerando testes escritos em linguagens de programação, testes baseados em especificação escritos em linguagens de domínio específico (*Domain-Specific Language* – DSL) e testes escritos em linguagem natural. O estudo foi conduzido em duas rodadas: a busca inicial, realizada em junho de 2024; e uma busca complementar, realizada em agosto de 2026, restrita às publicações posteriores à primeira rodada.

Nas duas rodadas foram utilizadas as bases ACM Digital Library, IEEE Xplore e Scopus. A expressão de busca foi (testing) AND (refactoring OR reuse) AND (automatic OR tool), adaptada a cada base. As restrições foram: (1) pesquisar em títulos, resumos e palavras-chave; (2) apenas trabalhos da área de Ciência da Computação; (3) apenas publicações em periódicos e conferências; e (4) apenas trabalhos em língua inglesa. A primeira rodada encontrou 933 publicações, sendo 59 da ACM Digital Library, 66 da IEEE Xplore e 808 da Scopus. A segunda encontrou 376 publicações, sendo 108 da ACM Digital Library, 125 da IEEE Xplore e 143 da Scopus. No total, as duas buscas retornaram 1.309 publicações: 167 da ACM Digital Library, 191 da IEEE Xplore e 951 da Scopus.

A seleção foi dividida em três etapas, aplicadas da mesma forma nas duas rodadas e consolidadas na Figura 10. Na primeira etapa, foram lidos títulos e resumos, e para cada publicação respondeu-se à pergunta: a publicação aborda refatoração ou reuso de casos de teste? O objetivo foi eliminar trabalhos sem relação direta com o tema, uma vez que os termos da expressão de busca podem aparecer em contextos distintos. A pergunta foi respondida com “Sim”, “Não” ou “Incerto”. Ao todo, 1.149 publicações receberam “Não” e foram descartadas; 56 receberam “Sim” e seguiram diretamente para a última etapa; e 104 receberam “Incerto” e foram avaliadas na segunda etapa.

Na segunda etapa, as 104 publicações incertas foram reavaliadas pela leitura da introdução e da conclusão. A mesma pergunta foi repetida, agora apenas com “Sim” ou “Não”. Dessas publicações, 100 receberam “Não” e foram descartadas, enquanto 4 receberam “Sim” e seguiram para a terceira etapa. Assim, 60 publicações foram lidas por completo, 56 provenientes diretamente da primeira etapa e 4 provenientes da segunda.

Na terceira etapa, respondeu-se à pergunta: a publicação trata de uma abordagem automática ou semiautomática de refatoração ou reuso de testes de software? Dezenove publicações foram consideradas relevantes e 41 foram excluídas. Os trabalhos excluídos abordam temas relacionados, mas não satisfazem o critério de inclusão por razões distintas. A abordagem de Qusef et al. (2023) adapta testes que deixam de funcionar após refatorações no código de aplicação, mas não refatora sua estrutura nem promove o reuso de código de teste. Maier e Felderer (2023) detectam *test smells*, mas não apresentam uma estratégia para transformar o código identificado. Os trabalhos de Veloso, Tsantalís e Chen (2026), Veloso (2026) e Soares, Ribeiro e Santos (2024) catalogam ou investigam empiricamente refatorações de teste, sem oferecer um mecanismo que as aplique de forma automática ou semiautomática. Zhao et al. (2025) investigam a prevalência de clones de *mocks* e avaliam sua remoção manual, enquanto Zhao (2025) descreve uma pesquisa em andamento sobre a futura automatização desse processo. Freitas et al. (2024) avaliam os efeitos de ações de teste reusáveis previamente definidas por engenheiros, mas não identificam nem extraem essas ações automaticamente. Teixeira, Silveira e Guerra (2025) empregam teste de mutação para verificar a preservação do comportamento após uma refatoração, sem realizar a transformação. Por fim, Heredia-Bravo et al. (2026) refatoram a infraestrutura de ferramentas de teste para adaptá-las a sistemas reais, e não o código ou os casos de teste. Assim, esses trabalhos foram excluídos porque apenas detectam, caracterizam, validam ou apoiam refatorações, porque dependem de transformações manuais ou porque refatoram artefatos diferentes daqueles considerados neste estudo.

Dos 19 registros que atenderam ao critério de inclusão, dois correspondiam a publicações recuperadas em mais de uma base de dados. Após a remoção dessas duplicatas, o estudo incluiu 17 publicações únicas. Os trabalhos selecionados mostram que a refatoração automática de testes está fragmentada entre objetivos e artefatos distintos. Alguns refatoram código de teste executável; outros, casos de teste em linguagem natural. Os objetivos também variam: substituição de *mocks*, remoção de *test smells*, melhoria de análise dinâmica ou reorganização de casos de teste. Essa diversidade confirma a relevância do tema, mas também indica que as abordagens existentes não oferecem uma arquitetura reusável para refatorar suítes completas com diferentes estratégias. Além disso, nenhuma delas trata a configuração implícita como um problema global sobre toda a suíte.

Três publicações relevantes pertencem ao mesmo grupo de autores (Wang et al., 2021; Wang et al., 2022; Wang et al., 2023). Esses trabalhos examinam o uso de herança para criar *mocks* de componentes que dependem de outros. Os autores argumentam que a herança não é o mecanismo mais adequado, pois introduz acoplamento com o código de aplicação. A proposta substitui automaticamente *mocks* baseados em herança por *mocks* de bibliotecas (e.g., Mockito²). Em avaliação com projetos de código aberto, a abordagem identificou 334 candidatos e refatorou 276 (83%). O foco, portanto, é substituir o mecanismo de *mock*, e não reduzir a duplicação do código de teste.

² <https://site.mockito.org/>

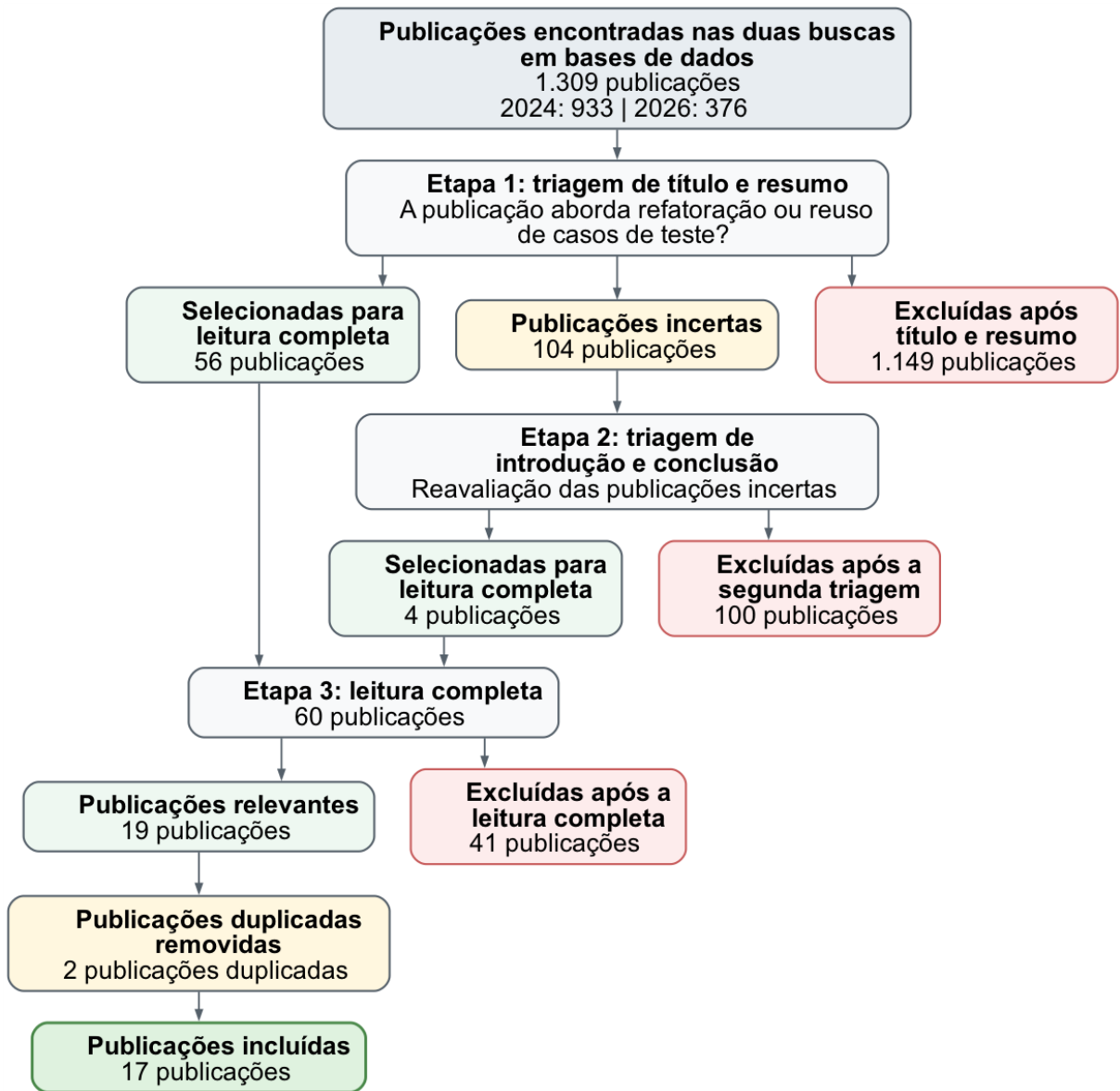


Figura 10 – Etapas consolidadas de filtragem das duas rodadas do estudo sistemático, das buscas iniciais até as 17 publicações incluídas

Fonte: Elaborado pelo autor.

Pizzini, Reinehr e Malucelli (2023a) propõem a refatoração automática de *eager test*. O principal desafio apontado é a detecção correta desse *test smell*, pois um método do SUT pode aparecer na preparação do teste e não no exercício. Para reduzir falsos positivos, o teste só é classificado como afetado quando invocações ao SUT intercalam-se com asserções. Em 350 classes de teste, 312 apresentavam o *test smell*; a abordagem refatorou 295 sem erros, e 17 falharam na execução. O tempo de execução aumentou 23%. Uma limitação é a necessidade de reordenar os métodos refatorados para que não falhem, o que evita duplicação ao custo de violar a independência dos testes (Meszaros, 2007). Em trabalho subsequente, Pizzini, Reinehr e Malucelli (2023b) apresentam o Sentinel, capaz de refatorar o *eager test* e a duplicação de código. Dada uma classe, o Sentinel busca linhas comuns entre os métodos, agrupa os testes

que as compartilham e os move para uma classe separada. A abordagem remove duplicação pela configuração implícita, porém opera sobre uma única classe.

Santana et al. (2022) apresentam refatorações de *assertion roulette* e *duplicate assert*. No primeiro, a ferramenta indica os trechos e o usuário fornece uma descrição para cada asserção, em um processo semiautomático. No segundo, o método de teste é dividido automaticamente em dois ou mais métodos. A extração pode introduzir duplicação, pois o código compartilhado é replicado. Em experimento, a refatoração manual de *assertion roulette* levou, em média, 78,30 segundos, contra 49,90 segundos com a ferramenta; para *duplicate assert*, os tempos foram 107,20 e 2,55 segundos.

Lambiase et al. (2020) apresentam o Dart, um plugin para o ambiente de desenvolvimento integrado (*Integrated Development Environment* – IDE) IntelliJ³, capaz de refatorar três *test smells*: *general fixture*, *eager test* e *lack of cohesion of test methods*. Diante de um *general fixture*, o Dart extrai o teste para uma nova classe cujo método de configuração implícita contém apenas os acessórios necessários. Para o *eager test*, decompõe o método de teste de modo que cada método resultante exercite um único método da classe de produção. Para a falta de coesão, agrupa e extrai métodos que cobrem as mesmas partes do código de aplicação. A detecção utiliza a ferramenta Taste, e o desenvolvedor decide se aplica a refatoração.

Xuan et al. (2016) apresentam o B-Refactoring, voltado à análise dinâmica (i.e., à análise do programa enquanto os testes executam). A ferramenta reestrutura o código de teste para melhorar a eficácia do Nopol, que repara falhas no código de aplicação sob a condição de que cada caso de teste produza um único rastro de execução. O B-Refactoring divide um teste em fragmentos menores, cada um exercitando um caminho. A refatoração é sob demanda: o código original não é substituído de forma permanente. O foco, portanto, é melhorar a análise dinâmica, e não a manutenibilidade do código de teste.

Shi, Huang e Wan (2022) propõem um método de reuso orientado a agrupamento para casos de teste escritos em linguagem natural. Palavras-chave são extraídas, uma matriz de similaridade é construída e aplica-se o algoritmo *spectral clustering* (Luxburg, 2007). O resultado são pacotes de casos de teste com alta similaridade semântica. A proposta não abrange o código de teste, apenas descrições em linguagem natural.

Bernard et al. (2020) apresentam abordagem semelhante, na qual casos de teste em linguagem natural são agrupados pelo escopo funcional. A ferramenta Orbiter combina passos que representam a mesma ação com redações distintas e parametriza passos que diferem apenas em valores. Em experimento, o número total de passos reduziu-se em 18%. Assim como Shi, Huang e Wan (2022), o trabalho não refatora o código de teste, apenas os casos descritos em linguagem natural.

Por fim, Hauptmann et al. (2015) examinam a refatoração de casos de teste em linguagem natural com técnicas de detecção de clones. Os casos são decompostos, trechos similares são extraídos e reusados, em comportamento análogo à configuração delegada. A abordagem é

³ <https://www.jetbrains.com/idea/>

especialmente útil em testes de sistema (e.g., procedimentos comuns de autenticação em testes de aceitação). O processo é semiautomático: a ferramenta refatora as descrições em linguagem natural, mas o código de teste correspondente ainda precisa ser alterado manualmente.

Gao et al. (2025) apresentam o UTRefactor, um *framework* baseado em modelos de linguagem para refatorar automaticamente *test smells* em testes unitários Java. A abordagem extrai contexto do código de teste e recorre a uma base externa com definições de *test smells* e regras de refatoração descritas em uma linguagem específica de domínio. O modelo é guiado por encadeamento de raciocínio, com um mecanismo de verificação para tratar múltiplos *test smells* no mesmo teste. Em 879 testes de seis projetos, o número de *test smells* caiu de 2.375 para 265 (89%). O UTRefactor superou a refatoração direta por modelo de linguagem em 61,82% na eliminação de *test smells* e também superou uma ferramenta baseada em regras. O foco, portanto, é remover *test smells* no interior dos testes, e não redistribuir testes entre classes para aumentar o reuso da configuração implícita.

Narang, Du e Jones (2025) apresentam a ferramenta *Untangle to Weave* – U2W, que detecta o *test smell* DAT por análise de programa, separa o método em testes focados e, em seguida, combina testes estruturalmente semelhantes em testes unitários parametrizados. Em 42.334 testes de 49 projetos, o DAT apareceu em 95,9% dos projetos, afetando em média 8,59% dos testes. Foram refatorados 3.638 testes; a redução média de linhas executáveis nos testes afetados foi de 36,33%. De 19 *pull requests* submetidos, 16 foram aceitos. Assim como o UTRefactor, a U2W opera sobre o interior dos métodos de teste e sobre clones locais, sem tratar a configuração implícita da suíte como um todo.

Derezińska e Sobieraj (2026) apresentam um plugin para o Eclipse, o TestSmellRefactorPlugin, voltado a programas Java com JUnit. O plugin detecta dez *test smells* (e.g., roleta de asserções, asserção duplicada, teste ansioso e teste de número mágico) e refatora os testes por análise da AST. O desenvolvedor pode aceitar cada transformação ou aplicar, de uma vez, todas as ocorrências de um *test smell* na classe. Em nove projetos, a detecção coincidiu em grande parte com a ferramenta tsDetect⁴, e os testes refatorados preservaram o resultado da execução. A abordagem é semiautomática e, como as anteriores, restringe-se a *test smells* no interior da classe de teste.

Guo et al. (2024) apresentam o PC-TRT, voltado ao reuso e à geração de casos de teste para cobertura de caminhos. A ferramenta compara caminhos da versão atual com caminhos cobertos por casos históricos: se a similaridade exceder um limiar, reusa-se o valor esperado do caso antigo; caso contrário, gera-se um caso novo por execução simbólica com o KLEE, ferramenta que explora caminhos do programa para gerar automaticamente entradas de teste. O objetivo é reduzir casos redundantes e preservar casos históricos de qualidade. O reuso, portanto, serve à cobertura estrutural, e não à manutenibilidade do código de teste.

Wang et al. (2024) propõem o ATAG, que gera uma arquitetura de teste a partir dos fluxos de controle e de dados dos casos existentes, de modo a recuperar funções de teste coesas

⁴ <https://testsmells.org/>

e pouco acopladas e, com isso, favorecer o reuso ao implementar novos casos. O FunBERT, baseado no *Bidirectional Encoder Representations from Transformers* – BERT, modelo de linguagem que representa o contexto das palavras nas duas direções da sentença, sugere nomes para essas funções. Em três conjuntos industriais, a arquitetura gerada melhorou, em média, o acoplamento em cerca de 26% a 35% e a coesão em cerca de 28% a 50% em relação ao desenho manual. O FunBERT obteve precisão de 97,9%, revocação de 98,3% e medida F1, média harmônica entre precisão e revocação, de 98,1%. A proposta reorganiza funções de teste para reuso, mas não refatora a configuração implícita da suíte.

Por fim, Almeida, Nogueira e Sampaio (2026) combinam casos de teste sequenciais de funcionalidades individuais para gerar testes ainda sequenciais que exercitam funcionalidades concorrentes. A abordagem inclui análise de dependência entre passos e uma relação de conformidade que considera a ausência de saída observável. Quando o modelo de casos de uso não está disponível, os novos testes são obtidos pela permutação de passos (*atoms*) dos casos existentes. Em avaliação com a Motorola Mobility, os testes gerados apresentaram cobertura e detecção de falhas superiores às da suíte produzida pelos engenheiros. O reuso, nesse caso, gera novos testes de concorrência a partir de testes já escritos, sem refatorar o código de teste em busca de manutenibilidade.

3.2.1 Discussão

As duas rodadas do estudo sistemático confirmam o quadro já esboçado na revisão exploratória. Há um conjunto diversificado de abordagens automáticas e semiautomáticas para refatorar ou reusar testes, porém nenhuma delas trata o problema que este trabalho formula. As técnicas encontradas operam sobre o interior de um método ou de uma classe, substituem mecanismos de *mock*, removem *test smells* pontuais, reorganizam descrições em linguagem natural ou reusam casos para cobertura e geração de novos testes. Mesmo quando extraem código comum para um método de configuração implícita, o recorte permanece local: o Sentinel restringe-se a uma única classe, e o Dart deixa ao desenvolvedor a decisão de aplicar a extração. Nenhuma abordagem redistribui os testes de uma suíte completa de modo a ampliar o reuso das sequências iniciais de configuração de acessórios.

Tampouco se identifica uma arquitetura reusável que separe as etapas de obtenção dos testes, medição de similaridade, agrupamento e refatoração, de forma que diferentes estratégias possam ser combinadas no mesmo fluxo. A similaridade, quando aparece, serve à redução de suíte, à priorização ou à recomendação de casos, e não à identificação de configurações de acessórios compartilhadas para reorganizar classes de teste.

Diante disso, a proposta deste trabalho é original em dois aspectos conjugados. Primeiro, trata a configuração implícita como um problema global de redistribuição de métodos entre classes de teste: métricas de similaridade e algoritmos de agrupamento são usados para considerar toda a suíte, de modo a reduzir a duplicação sem alterar o resultado observado dos testes. Segundo, materializa essa abordagem em uma arquitetura em *pipeline* e no *framework*

Róza⁵, que instancia essa arquitetura com foco na estratégia de configuração implícita e admite a incorporação de outras estratégias. Nenhum dos trabalhos recuperados combina esses dois elementos.

⁵ <https://github.com/lucasPereira/roza>

4 PROPOSTA

Conforme apresentado no Capítulo 1, a duplicação do código de configuração de acessórios prejudica a manutenibilidade dos testes, mas sua remoção manual torna-se progressivamente mais difícil à medida que a suíte cresce. O problema não se limita a identificar trechos repetidos: também é necessário decidir como redistribuir os métodos entre classes de teste sem aumentar o esforço total de desenvolvimento e manutenção e sem alterar o resultado observado dos testes.

O objetivo deste trabalho consiste em reduzir essa duplicação agrupando, em uma mesma classe, testes que possuam a mesma sequência inicial de configuração de acessórios. O código comum pode, então, ser extraído para um método de configuração executado antes de cada teste e reusado por meio da estratégia de configuração implícita. Dessa forma, cada método mantém apenas a configuração que lhe é específica, enquanto a classe centraliza a parte compartilhada por todos os seus testes.

Essa distribuição não é tratada como uma sucessão de decisões locais, nas quais se escolhe isoladamente uma classe para cada novo teste. A movimentação de um teste pode alterar tanto o código reusável quanto a duplicação introduzida nos demais testes da classe. Por isso, este trabalho aborda a distribuição dos testes como um problema global de redistribuição de métodos entre classes: busca-se uma organização que considere toda a suíte e reduza a duplicação ao aumentar o reuso das sequências iniciais de configuração de acessórios, preservando o comportamento observado dos testes. A refatoração pode ser aplicada em qualquer momento da evolução da suíte. Assim, quando novos testes forem adicionados, o processo pode ser executado novamente para recalcular a distribuição global e incorporá-los à organização das classes.

Para operacionalizar essa abordagem, o trabalho contribui com (1) uma arquitetura em *pipeline* para refatoração de código de teste; e (2) o Róza, um *framework* extensível que instancia essa arquitetura. A arquitetura define as responsabilidades e o fluxo de dados da refatoração, enquanto o *framework* oferece componentes e pontos de extensão para concretizar esse fluxo. As seções seguintes apresentam esses dois elementos.

4.1 ARQUITETURA EM PIPELINE

A Figura 11 apresenta a arquitetura, que organiza o processo de refatoração em um *pipeline* de sete etapas: carregamento, análise sintática, decomposição, medição, agrupamento, refatoração e escrita. Cada etapa recebe os dados produzidos pela etapa anterior e gera a entrada da próxima. Essa separação permite substituir uma estratégia sem modificar as demais partes do processo. Assim, diferentes formas de obter os arquivos, linguagens, *frameworks* de teste, métricas de similaridade, algoritmos de agrupamento, estratégias de refatoração e destinos de escrita podem ser combinados dentro do mesmo fluxo. Embora o objetivo deste trabalho seja promover o reuso de código por meio da configuração implícita, a arquitetura não se res-

tringe a essa estratégia. Sua flexibilidade permite incorporar outros tipos de refatoração, como a extração de código comum por meio da configuração delegada.



Figura 11 – Arquitetura em *pipeline* para refatoração de código de teste

Fonte: Elaborado pelo autor.

O fluxo de dados percorre as sete etapas na seguinte ordem: (1) o carregamento produz os arquivos de código; (2) a análise sintática identifica as classes de teste e as converte em um modelo estrutural em memória; (3) a decomposição produz uma representação individual de cada teste; (4) a medição produz a matriz de similaridade entre os testes; (5) o agrupamento produz grupos de testes; (6) a refatoração aplica uma estratégia a esses grupos e produz o código de teste refatorado; e (7) a escrita persiste os arquivos de código refatorados.

A primeira etapa, denominada **carregamento**, obtém os arquivos de código e os mantém em memória. Esses arquivos ainda não são interpretados como testes: o carregamento não identifica classes de teste nem constrói um modelo estrutural. Essa separação permite combinar diferentes estratégias de obtenção de arquivos (e.g., carregamento a partir do sistema de arquivos ou a partir de um repositório remoto) com as etapas seguintes.

Na etapa de **análise sintática**, os arquivos carregados são examinados para identificar aqueles que contêm classes de teste. Cada classe identificada é convertida em uma árvore abstrata de sintaxe (*Abstract Syntax Tree* – AST), que representa sua estrutura em memória e permite identificar e manipular atributos, métodos de configuração e métodos de teste.

Na etapa de **decomposição**, cada teste é separado de sua classe de origem e associado a todo o código necessário para sua execução. A decomposição realiza, portanto, o movimento oposto ao da refatoração: em vez de agrupar testes e extrair o código comum, ela incorpora a cada teste a configuração da qual ele depende para isolá-lo de sua classe original. Isso inclui tanto a configuração declarada no próprio método quanto aquela compartilhada na classe por configuração implícita. O resultado é uma representação individual de cada teste, composta por sua configuração de acessórios, seu exercício e suas asserções. Esse isolamento permite que os testes sejam comparados individualmente na etapa de medição.

Na etapa de **medição**, consideram-se apenas as configurações de acessórios, pois são elas que contêm o código que se pretende reusar. A métrica calcula o grau de similaridade entre cada par de testes e produz uma matriz de similaridade. Quando a métrica opera sobre arquivos, a configuração de cada teste é materializada internamente nesta etapa, isto é, gravada em um arquivo separado. Métricas capazes de consumir a representação em memória dispensam

essa materialização. A matriz reúne as relações entre todos os testes da suíte e constitui a entrada da etapa de agrupamento. Embora a arquitetura permita substituir a métrica de forma independente, é desejável que ela esteja alinhada à estratégia de refatoração aplicada nas etapas seguintes. Isso porque o agrupamento opera sobre os graus produzidos pela métrica: se esses graus privilegiarem trechos comuns que a estratégia não consegue extrair, os grupos formados não reduzirão a duplicação.

Na etapa de **agrupamento**, um algoritmo utiliza a matriz de similaridade para propor uma distribuição global dos testes. Cada grupo constitui a unidade sobre a qual a etapa de refatoração atua. O objetivo não é apenas reunir os pares mais similares isoladamente, mas encontrar grupos cuja similaridade, segundo o critério da métrica, possibilite reduzir a duplicação total da suíte.

Na etapa de **refatoração**, o código de teste é reconstruído a partir dos grupos propostos. Para cada grupo, aplica-se uma estratégia de refatoração que extrai o código comum segundo o contrato dessa estratégia. As métricas de similaridade e os algoritmos de agrupamento permitem encontrar grupos de testes com código comum. Entretanto, essa identificação não garante a qualidade da refatoração: é preciso que a estratégia consiga reduzir a duplicação dos trechos encontrados. A etapa não prescreve uma única transformação: diferentes estratégias podem ser combinadas no mesmo fluxo (e.g., extração para um método de configuração implícita ou extração para métodos de configuração delegada). Qualquer que seja a estratégia, a transformação deve preservar o comportamento observado dos testes. A transformação assume a independência dos testes, discutida no Capítulo 2: os acessórios de um teste não devem depender da ordem de execução nem do estado residual deixado por outros testes.

Por fim, a etapa de **escrita** persiste os arquivos de código refatorados em um destino de saída, como o sistema de arquivos. Com isso, o fluxo proposto se encerra em artefatos de código, e não apenas em um modelo em memória.

4.2 FRAMEWORK RÓZA

O Róza é um *framework* extensível que constitui uma instância da arquitetura em *pipeline*, com foco na refatoração por meio da estratégia de configuração implícita, e encontra-se em desenvolvimento. No estágio atual desta pesquisa, foi implementado o fluxo até a etapa de medição, compreendendo o carregamento, a análise sintática, a decomposição e a medição. Esse fluxo produz a matriz de similaridade necessária para a continuação do *pipeline*. As etapas de agrupamento, refatoração e escrita integram a arquitetura proposta, mas ainda não foram implementadas no *framework*. A Figura 12 apresenta as classes e as interfaces atualmente implementadas, organizadas em quatro pacotes que correspondem às etapas já concretizadas. Por concisão, o diagrama omite a maior parte das demais classes e apresenta apenas as principais.

No carregamento, a interface `CodeFileLoader` produz `LoadedCodeFiles`, um conjunto de `CodeFile`. Cada arquivo conserva a origem (e.g., o caminho no sistema de arquivos) e o conteúdo textual, ainda sem interpretação como teste. Na análise sintática, a interface

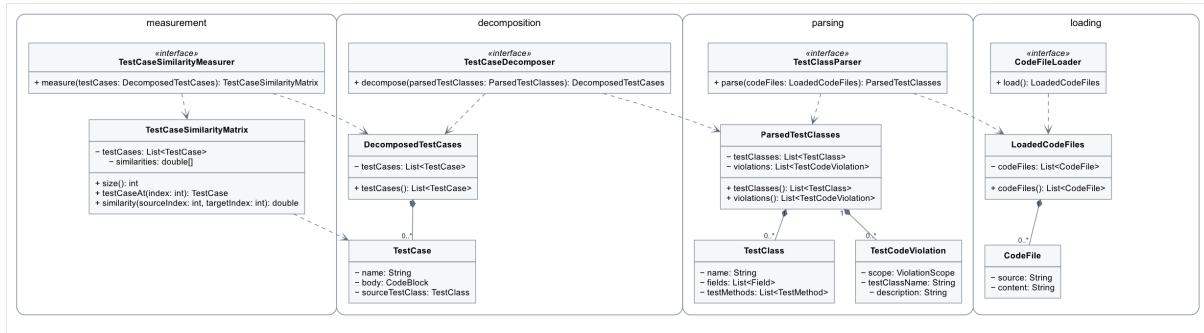


Figura 12 – Diagrama de classes do Róza

Fonte: Elaborado pelo autor.

`TestClassParser` interpreta os arquivos carregados. O artefato `ParsedTestClasses` reúne as classes de teste identificadas, representadas por `TestClass`, com nome, atributos e métodos de teste, e as violações, representadas por `TestCodeViolation`, registradas quando uma restrição impede a refatoração. Na decomposição, a interface `TestCaseDecomposer` isola cada teste. O artefato `DecomposedTestCases` contém um `TestCase` para cada método, com nome, corpo (configuração de acessórios, exercício e asserções) e referência à classe de origem. Na medição, a interface `TestCaseSimilarityMeasurer` produz a matriz de similaridade. O artefato `TestCaseSimilarityMatrix` registra o grau de similaridade entre cada par de testes decompostos.

Cada etapa é representada por um contrato de entrada e de saída, e não por uma implementação rígida. As implementações não se invocam mutuamente: um orquestrador consome o artefato produzido por uma etapa e o fornece à seguinte. Com isso, uma implementação de análise sintática pode ser combinada com diferentes implementações de medição sem que qualquer uma delas conheça a outra. Novas linguagens ou métricas de similaridade entram no fluxo pela substituição do contrato correspondente.

A Figura 13 apresenta a interface gráfica do Róza na etapa de decomposição. À esquerda, o resumo das violações registra a contagem de classes e testes afetados, os testes excluídos e os testes aceitos no *pipeline*. À direita, a aba *Classes* permite inspecionar cada classe de teste e consultar, na aba *Summary*, atributos, configurações, testes e violações associadas.

4.2.1 Análise sintática

A implementação atual processa classes de teste escritas em Java com o *framework* `JUnit`, nas versões 4 e 5. A análise sintática dessas classes é realizada com a biblioteca `JavaParser`¹, da qual resulta o modelo estrutural utilizado nas etapas seguintes.

A análise sintática não se limita a construir o modelo estrutural. O analisador também busca construções que não são tratadas pelo Róza ou cuja preservação não pode ser assegurada

¹ <https://javaparser.org/>

Fonte: Elaborado pelo autor.

Entre outros requisitos, uma classe de teste deve ser concreta, não utilizar herança nem declarar tipos aninhados, conter métodos de teste sem parâmetros e possuir, no máximo, um método de configuração implícita (`@Before` ou `@BeforeEach`). As violações são registradas por cinco razões, que podem ocorrer simultaneamente: (1) a construção constitui uma má prática; (2) o comportamento que deveria ser preservado não está suficientemente definido para a refatoração; (3) o suporte à construção foi excluído para simplificar a implementação do Róza; (4) a transformação pode alterar a compilação ou o resultado observado dos testes; e (5) não há uma forma coerente de representar ou refatorar a construção no modelo adotado. O Apêndice A descreve cada violação e indica as razões aplicáveis.

4.2.2 Medição

O *framework* oferece cinco métricas de similaridade: JPlag (Prechelt; Malpohl; Philippsen, 2002)², Simian³, Deckard (Jiang et al., 2007)⁴, a subsequência comum mais longa (*Longest Common Subsequence* – LCS) (Hirschberg, 1977) e a subsequência comum inicial contígua mais longa (*Longest Common Contiguous Start Subsequence* – LCCSS) (Silva; Vilain, 2020). Essas métricas cobrem as abordagens baseadas em *tokens*, em texto e em árvores, discutidas no Capítulo 2, além de duas métricas sobre sequências de comandos. As seções seguintes apresentam cada uma dessas métricas. JPlag, Simian e Deckard são ferramentas existentes de detecção de clones. O Róza não as reimplementa: para cada uma, oferece um adaptador que invoca a ferramenta e interpreta a saída produzida, a fim de obter o grau de similaridade e registrá-lo na matriz.

4.2.2.1 JPlag

JPlag consiste em um detector de clones baseado em *tokens*, com suporte a linguagens como Java, C++ e Python (Prechelt; Malpohl; Philippsen, 2002). A ferramenta admite um parâmetro de sensibilidade, correspondente ao número mínimo de *tokens* que devem coincidir para que um trecho seja considerado comum. No Róza, o adaptador materializa internamente a configuração de acessórios de cada teste em arquivos, invoca a ferramenta e interpreta o relatório (*HyperText Markup Language* – HTML) para obter o grau de similaridade de cada par. O relatório informa dois graus, um para cada sentido da comparação, o que torna a métrica assimétrica (i.e., $\delta(\alpha, \beta)$ pode diferir de $\delta(\beta, \alpha)$).

4.2.2.2 Simian

Simian consiste em um detector de clones comercial baseado em texto, aplicável a linguagens como Java, C++, C# e Ruby. A ferramenta produz um relatório dos fragmentos considerados clones, e não um grau de similaridade. Por isso, o adaptador interpreta essa saída e calcula o grau de similaridade conforme a Equação 4.1, definida como a razão entre o tamanho total dos fragmentos clonados, medido em linhas, e o tamanho da configuração de acessórios de origem (Ragkhitwetsagul; Krinke; Clark, 2018). Como as configurações podem ter tamanhos distintos, a métrica é assimétrica (i.e., $\delta(\alpha, \beta)$ pode diferir de $\delta(\beta, \alpha)$). Fragmentos sobrepostos são unidos para que não sejam contados mais de uma vez. Simian admite um limiar correspondente ao número mínimo de linhas semelhantes, que deve ser um inteiro positivo com valor mínimo 2. Assim como em JPlag, a configuração de acessórios é materializada internamente em arquivos.

² <https://jplag.de>

³ <https://simian.quandarypeak.com/>

⁴ <https://github.com/skyhover/Deckard>

$$\delta(\alpha, \beta) = \frac{\sum_{i=1}^n |\text{FRAGMENT}_i^\alpha(\alpha, \beta)|}{|\alpha|} \quad (4.1)$$

Na Equação 4.1, α e β denotam as configurações de acessórios de dois testes. $|\alpha|$ é o número de linhas da configuração de α . $\text{FRAGMENT}_i^\alpha(\alpha, \beta)$ é o i -ésimo fragmento de α identificado como clone em relação a β , e $|\text{FRAGMENT}_i^\alpha(\alpha, \beta)|$ é o número de linhas desse fragmento após a união dos intervalos sobrepostos.

4.2.2.3 Deckard

Deckard consiste em um detector de clones baseado em árvores (Jiang et al., 2007). Assim como Simian, a ferramenta produz um relatório de clones. O adaptador interpreta essa saída e obtém o grau de similaridade pela Equação 4.1, o que também torna a métrica assimétrica. Deckard oferece três parâmetros: o número mínimo de *tokens* para classificar dois fragmentos como clones; o passo (STRIDE), que indica quantos *tokens* podem ser ignorados ao unir fragmentos não contíguos; e o limiar mínimo de similaridade, com valores entre 0,0 e 1,0. Como a ferramenta opera sobre arquivos, o adaptador materializa internamente a configuração de acessórios de cada teste antes de invocá-la.

4.2.2.4 LCS

A LCS não é, em si, uma métrica de similaridade, mas uma sequência obtida a partir de outras duas, contendo os elementos comuns e preservando a ordem original (Hirschberg, 1977). No Róza, a LCS foi convertida em uma métrica de similaridade por meio da representação de cada teste como uma sequência de comandos da configuração de acessórios, e essa métrica foi implementada no *framework*. O grau de similaridade é calculado pela Equação 4.2, baseada no coeficiente de Dice (1945). A métrica é simétrica e não exige configuração. Diferentemente de JPlag, Simian e Deckard, a LCS consome a representação em memória e dispensa a materialização em arquivos.

$$\delta(\alpha, \beta) = \frac{|\text{LCS}(\alpha, \beta)| \times 2}{|\alpha| + |\beta|} \quad (4.2)$$

Na Equação 4.2, $|\alpha|$ e $|\beta|$ correspondem ao número de comandos da configuração de acessórios de cada teste, e $|\text{LCS}(\alpha, \beta)|$ corresponde ao número de comandos da subsequência comum mais longa. A normalização pelo tamanho das duas sequências produz um grau entre 0 e 1.

A Figura 14 ilustra a LCS por meio de dois testes sobre um pedido. As sequências de comandos são $\alpha = \langle A, B, C, D, E \rangle$ e $\beta = \langle A, B, X, D, Y \rangle$. Os comandos A e B criam o pedido e adicionam o primeiro item; C aplica um desconto; X adiciona outro item; D confirma o pedido; e E e Y verificam os respectivos totais. A LCS contém A , B e D , pois preserva a ordem original

Teste α		Teste β		Legenda
pedido = Pedido()	A	pedido = Pedido()	A	prefixo comum
pedido.adicionar(livro)	B	pedido.adicionar(livro)	B	comandos diferentes
pedido.aplicarDesconto(10)	C	pedido.adicionar(caneta)	X	comum após divergência
pedido.confirmar()	D	pedido.confirmar()	D	
assertEquals(90, total())	E	assertEquals(120, total())	Y	

LCS(α, β)			
A	B	D	$\delta = 0,60$

Figura 14 – Subsequência comum mais longa entre dois testes

Fonte: Elaborado pelo autor.

e admite o comando comum posterior à divergência. Com $|LCS(\alpha, \beta)| = 3$ e $|\alpha| + |\beta| = 10$, a Equação 4.2 atribui grau 0,60.

4.2.2.5 LCCSS

A subsequência comum inicial contígua mais longa (*Longest Common Contiguous Start Subsequence*, LCCSS) constitui uma das contribuições desta pesquisa (Silva; Vilain, 2020). Foi proposta para identificar pares de testes candidatos à refatoração por meio da configuração implícita. Seu desenvolvimento partiu da constatação de que métricas gerais de similaridade de código podem favorecer trechos comuns localizados em qualquer parte dos testes, embora a configuração implícita permita extrair somente uma sequência comum situada no início de todos os métodos da classe.

A LCCSS foi inspirada na LCS, mas restringe a subsequência comum a um prefixo contíguo. Se $\alpha = \langle a_1, \dots, a_m \rangle$ e $\beta = \langle b_1, \dots, b_n \rangle$ representam as sequências de comandos da configuração de acessórios de dois testes, então $LCCSS(\alpha, \beta)$ corresponde à maior sequência $\langle c_1, \dots, c_k \rangle$ para a qual $c_i = a_i = b_i$ em todas as posições de 1 a k . Dois comandos são considerados iguais quando coincidem após a normalização, que remove comentários e espaços em branco irrelevantes, sem abstrair identificadores nem literais. A comparação começa no primeiro comando e termina assim que ocorre a primeira divergência. Consequentemente, comandos iguais encontrados depois dessa divergência não integram a LCCSS.

$$\delta(\alpha, \beta) = \frac{|LCCSS(\alpha, \beta)| \times 2}{|\alpha| + |\beta|} \quad (4.3)$$

Na Equação 4.3, $|\alpha|$ e $|\beta|$ correspondem ao número de comandos das duas configurações de acessórios, e $|LCCSS(\alpha, \beta)|$ corresponde ao comprimento do prefixo comum contíguo (i.e., ao número de comandos que podem ser movidos com segurança para o método de confi-

guração implícita). O grau de similaridade é obtido pela mesma normalização da Equação 4.2 e expressa a proporção extraível em relação ao tamanho das duas configurações: vale 1 quando as configurações são idênticas e diminui conforme o prefixo comum encurta; vale 0 somente quando não há prefixo comum (i.e., quando o primeiro comando já diverge). A métrica é simétrica, não exige parâmetros de configuração e opera diretamente sobre a representação dos testes em memória. A comparação percorre as sequências até a primeira divergência e possui complexidade linear no comprimento da menor delas, em contraste com a obtenção da LCS, que é quadrática.

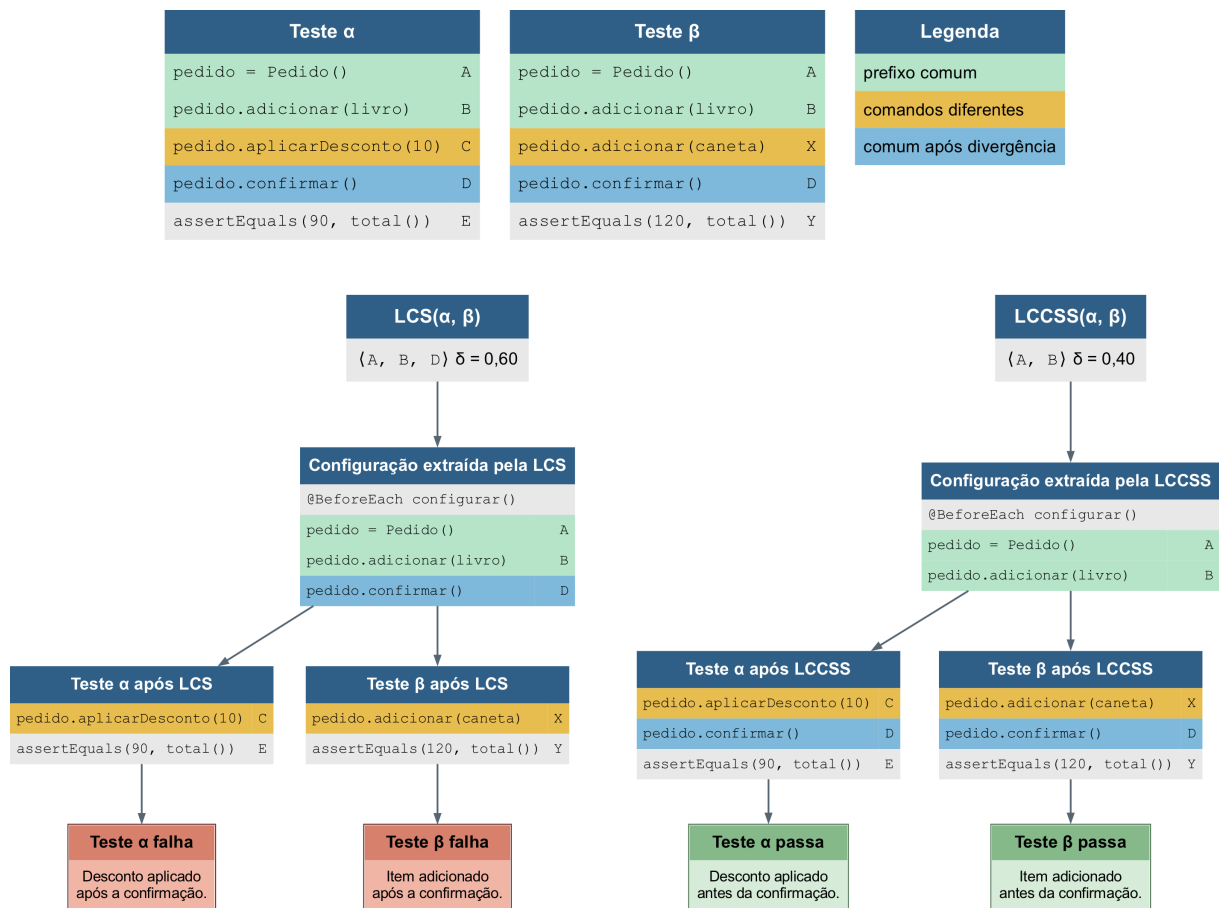


Figura 15 – Comparação entre a LCS e a LCCSS e efeito da extração de um comando não contíguo

Fonte: Elaborado pelo autor.

A Figura 15 retoma os testes da Figura 14 e contrapõe a extração indicada pela LCS à extração indicada pela LCCSS. A LCS contém A , B e D , pois admite o comando comum posterior à divergência, e atribui grau 0,60. A LCCSS contém somente o prefixo $\langle A, B \rangle$ e atribui grau 0,40.

A parte inferior da figura contrapõe os efeitos das extrações indicadas pelas duas métricas. No ramo da LCS, A , B e D são movidos para o método de configuração implícita. Com isso, a confirmação do pedido é executada antes de C e X : o teste α tenta aplicar um desconto a um pedido já confirmado, enquanto o teste β tenta adicionar um item a esse pedido, e ambos falham. No ramo da LCCSS, somente o prefixo comum contíguo $\langle A, B \rangle$ é extraído. O comando

D permanece nos métodos de origem, depois de C e X , o que preserva a ordem dos comandos e permite que os dois testes passem.

A restrição de prefixo aparece em um caso complementar. Se as sequências fossem $\alpha = \langle A, B, C, D \rangle$ e $\beta = \langle X, B, C, D \rangle$, a LCS conteria B , C e D e atribuiria grau 0,75, embora nenhum comando pudesse ser extraído para a configuração implícita, pois a sequência compartilhada não ocupa o início dos métodos. Nesse caso, $LCCSS(\alpha, \beta) = \langle \rangle$ e o grau é 0. A configuração implícita não se aplica a esse par; outra estratégia (e.g., testes parametrizados) seria mais adequada.

4.2.2.6 Comparação das métricas

A Tabela 1 sintetiza as cinco métricas. JPlag, Simian e Deckard operam sobre arquivos, exigem parâmetros e são assimétricas, pois o grau depende do tamanho da configuração de origem. A LCS e a LCCSS operam sobre a representação em memória, não exigem configuração e são simétricas. A última coluna indica se a métrica atribui grau elevado quando a sequência inicial das configurações coincide, condição necessária para mover o código comum ao método de configuração implícita. Apenas a LCCSS favorece esse prefixo: as demais podem atribuir grau elevado a trechos comuns que não ocupam o início das configurações e, portanto, não poderiam ser extraídos com segurança.

Tabela 1 – Comparação das métricas de similaridade oferecidas pelo Róza

Métrica	Abordagem	Simetria	Representação	Parâmetros	Prefixo comum
JPlag	<i>tokens</i>	assimétrica	arquivos	sensibilidade	Não
Simian	texto	assimétrica	arquivos	limiar de linhas	Não
Deckard	árvores	assimétrica	arquivos	<i>tokens</i> , passo e limiar	Não
LCS	comandos	simétrica	memória	nenhum	Não
LCCSS	comandos	simétrica	memória	nenhum	Sim

4.2.3 Etapas ainda não implementadas

As cinco métricas produzem a matriz de similaridade que constitui a entrada da etapa de agrupamento. As etapas de agrupamento, refatoração e escrita permanecem definidas na arquitetura e ainda não foram implementadas no Róza. O agrupamento deverá converter a matriz em grupos de testes; a refatoração, reconstruir as classes com configuração implícita a partir desses grupos; e a escrita, persistir os arquivos resultantes. A avaliação da abordagem quanto à redução de duplicação e ao esforço de refatoração, enunciada entre os objetivos específicos, é tratada no Capítulo 5.

5 AVALIAÇÃO

Conforme apresentado no Capítulo 4, a etapa de medição do *pipeline* produz uma matriz de similaridade entre os testes decompostos. Essa matriz constitui a base para identificar pares de testes candidatos à refatoração por meio da configuração implícita e, posteriormente, para o agrupamento global da suíte. Antes que o agrupamento e a refatoração automática possam ser avaliados de ponta a ponta no Róza, é necessário verificar se as métricas de similaridade recuperam, com precisão aceitável, os pares de testes que de fato podem compartilhar código de configuração de acessórios.

Este capítulo apresenta o experimento conduzido para comparar as cinco métricas disponíveis no Róza (JPlag, Simian, Deckard, LCS e LCCSS) quanto à sua capacidade de identificar pares de testes adequados à refatoração por configuração implícita. O experimento foi originalmente relatado em Silva e Vilain (2020) e reúne, nesta versão do trabalho, a metodologia, os resultados e as ameaças à validade alinhados à terminologia e à arquitetura em *pipeline* adotadas aqui. A avaliação da redução de duplicação e do esforço de refatoração após o agrupamento e a reconstrução das classes de teste permanece dependente da implementação das etapas subsequentes do *pipeline*; o experimento aqui descrito constitui, portanto, a avaliação empírica da etapa de medição, inclusive da métrica LCCSS.

5.1 OBJETIVO E DESENHO DO EXPERIMENTO

O objetivo do experimento consiste em comparar métricas de similaridade quanto à sua eficácia em recuperar pares de testes classificados manualmente como candidatos à refatoração por configuração implícita. Para isso, adota-se a perspectiva de recuperação da informação (Baeza-yates; Ribeiro-neto et al., 1999): para cada teste da suíte, a métrica em avaliação produz uma ordenação dos demais testes por grau de similaridade decrescente; essa ordenação é confrontada com um conjunto de referência de pares compatíveis, elaborado pelos autores por meio de classificação manual.

O experimento divide-se em cinco fases, executadas em sequência: (1) seleção da fonte de dados; (2) classificação manual e construção da matriz de controle; (3) geração de variantes de configuração para cada métrica; (4) execução de cada variante e obtenção das matrizes de similaridade; e (5) coleta dos resultados por meio de curvas médias de precisão e recaptção (*precision/recall*). As fontes, os *scripts* e as instruções de replicação encontram-se no repositório público do Róza^{1,2}.

A Figura 16 sintetiza esse fluxo e explicita os artefatos transferidos entre as fases. Na primeira fase, a suíte de testes de aceitação é filtrada segundo critérios de compatibilidade com o processamento do Róza, resultando no conjunto de 46 testes que alimenta as etapas seguintes.

¹ <https://github.com/lucasPereira/roza/tree/master/experiment-results>

² <https://github.com/lucasPereira/roza/tree/master/src/expt>

Na segunda fase, os autores classificam manualmente cada um dos 1035 pares distintos formados por esses testes e constroem a matriz de controle binária, na qual cada célula indica se a refatoração por configuração implícita seria adequada (1) ou inadequada (0); a partir dessa matriz, derivam-se os conjuntos de testes compatíveis que servirão de referência na avaliação.

Na terceira fase, após a construção da matriz de controle, definem-se as variantes de configuração de cada métrica. JPlag, Simian e Deckard admitem parâmetros de sensibilidade que alteram a detecção de trechos semelhantes; a LCS e a LCCSS não exigem ajuste e constituem variantes únicas. O diagrama resume o cardinal dessas variantes: 51 para JPlag, 14 para Simian, 476 para Deckard, 1 para LCS e 1 para LCCSS.

Na quarta fase, cada variante é aplicada ao conjunto de testes selecionado, produzindo uma matriz de similaridade com graus numéricos entre todos os pares de testes. A partir de cada matriz, gera-se, para cada teste, uma ordenação dos demais testes por grau de similaridade decrescente.

Na quinta fase, confrontam-se as ordenações produzidas por cada variante com os conjuntos de testes compatíveis da matriz de controle e calculam-se curvas médias de precisão e recaptação; para cada métrica, seleciona-se a variante de melhor desempenho para a análise comparativa final.

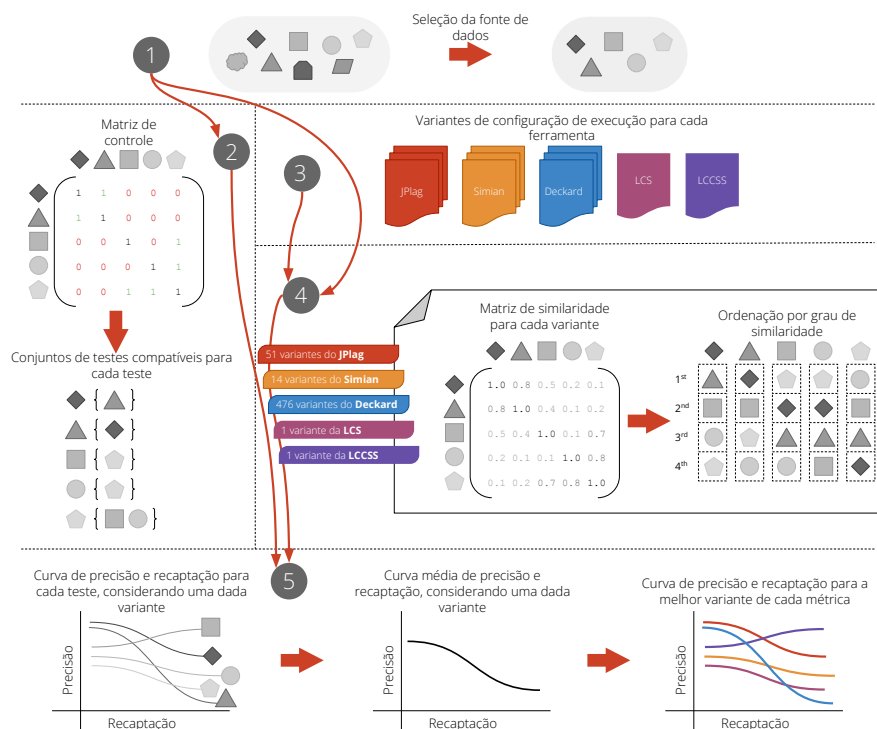


Figura 16 – Fases do experimento de avaliação das métricas de similaridade

Fonte: Elaborado pelo autor.

5.2 FONTE DE DADOS

A fonte de dados consiste em 46 testes de aceitação em nível de sistema, escritos em Java com JUnit e extraídos de um sistema para avaliação de produções de livros desenvolvido para a Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES). Esse sistema é um *fork* do projeto de código aberto *Open Monograph Press* (OMP)³. Selecionaram-se os testes do *fork* por serem compatíveis com o processamento do Róza, enquanto os testes originais do OMP, escritos em PHP, foram excluídos. Os 46 testes utilizam Selenium⁴ para controlar o navegador e estão distribuídos em seis classes de teste, com média de 209 linhas de código por classe.

5.3 MATRIZ DE CONTROLE

Na segunda fase, os 1035 pares distintos formados pelos 46 testes foram classificados manualmente. Para cada par, respondeu-se à seguinte pergunta: é adequado colocar os dois testes na mesma classe de teste a fim de reusar código de configuração de acessórios por meio da configuração implícita? Por adequado, entende-se que o reuso obtido por refatoração manual seria relevante o suficiente para compensar o esforço de reorganização.

Dessa análise resultaram 219 pares classificados como adequados à refatoração por configuração implícita e 816 pares classificados como inadequados. Para reduzir viés e erro humano, a classificação foi realizada duas vezes e os casos conflitantes foram reanalisados. A partir da matriz binária de controle, gerou-se, para cada teste, o conjunto de testes compatíveis (i.e., os demais testes com os quais poderia compartilhar um método de configuração implícita). Esses conjuntos alimentam a quinta fase do experimento.

5.4 VARIANTES DE CONFIGURAÇÃO

JPlag, Simian e Deckard admitem parâmetros que alteram a sensibilidade da detecção de trechos semelhantes; a LCS e a LCCSS não exigem configuração. Para assegurar comparação justa entre as métricas parametrizáveis, gerou-se o produto cartesiano dos valores candidatos de cada parâmetro, conforme a Tabela 2. A variante com melhor desempenho médio de precisão nos onze níveis padrão de recaptação foi selecionada para representar cada métrica na análise final.

Os intervalos de valores tomaram como referência os padrões de cada ferramenta e os valores empregados por Ragkhitwetsagul, Krinke e Clark (2018) em experimentos com detectores de clones.

³ <https://github.com/pkp/omp>

⁴ <https://selenium.dev/>

Tabela 2 – Parâmetros e valores utilizados na geração de variantes de configuração

Métrica	Parâmetro	Valores	Variantes
JPlag	sensibilidade (-t)	1 a 50	51
Simian	limiar de linhas (-threshold)	2 a 15	14
Deckard	MIN_TOKENS	1 a 15	476
	STRIDE	0 a 15 e inf	
	SIMILARITY	0,9 e 1,0	
LCS	nenhum	—	1
LCCSS	nenhum	—	1

5.5 EXECUÇÃO

Na quarta fase, cada variante de configuração foi aplicada aos 46 testes por meio do Róza, produzindo uma matriz de similaridade simétrica ou assimétrica conforme a métrica. A partir de cada matriz, obteve-se, para cada teste, o grau de similaridade de cada candidato; ordenados do maior para o menor, esses valores constituem a saída comparada com os conjuntos de testes compatíveis definidos na matriz de controle.

5.6 MÉTRICAS DE AVALIAÇÃO E RESULTADOS

Na quinta fase, confrontam-se as ordenações produzidas por cada variante com o gabarito definido na segunda fase. O procedimento pode ser lido em três etapas sucessivas: primeiro, para cada um dos 46 testes de referência, gera-se uma curva de precisão e recaptção a partir da ordenação da variante em relação a esse teste; em seguida, tira-se a média dessas 46 curvas, obtendo a curva da variante; por fim, escolhe-se a melhor variante dentro de cada métrica.

Para cada teste de referência T , o gabarito é um conjunto $C(T)$ de testes compatíveis. A variante em avaliação, por sua vez, atribui um grau de similaridade a cada candidato e produz uma ordenação dos demais testes, do maior para o menor grau. A avaliação verifica se os candidatos que a métrica coloca no topo dessa ordenação coincidem com $C(T)$.

A geração das curvas percorre a ordenação do candidato mais similar ao menos similar. Cada nível padronizado de recaptção (0%, 10%, ..., 100%) corresponde a uma fração dos compatíveis que deve ser recuperada: no nível de 10%, por exemplo, exige-se recuperar 10% dos elementos de $C(T)$; no de 50%, metade deles. Calcula-se, para cada nível, quantos elementos de $C(T)$ essa fração representa; a ordenação é percorrida candidato a candidato, do topo em diante, até recuperar essa quantidade. Com o prefixo definido, a precisão é a proporção de candidatos recuperados que o gabarito classifica como compatíveis. Isso produz, para cada teste, onze pares de recaptção e precisão.

A agregação seguinte condensa as 46 curvas individuais em uma curva por variante. Para cada nível de recaptção, calcula-se a média aritmética das 46 precisões observadas na-

aquele nível. Em outras palavras, fixa-se a recaptção (por exemplo, 50%) e tira-se a média das precisões que os 46 testes atingiram nesse nível; repete-se para os onze níveis.

O exemplo a seguir ilustra o cálculo para o teste `importarProducoesComSucesso` com a variante JPlag de sensibilidade 23. Para esse teste, o gabarito contém oito testes compatíveis, todos da mesma funcionalidade de importação de produções. Já a ordenação produzida pela métrica coloca, no topo, testes de outras funcionalidades com grau elevado (e.g., `importarAvaliadoresCoordenadorComSucesso`); por isso, sua precisão é penalizada. Por exemplo, considerando o nível de recaptção de 20%, é necessário recuperar um teste compatível ($\lfloor 0,2 \times 8 \rfloor = 1$); o menor prefixo que atinge essa meta na variante tem 16 candidatos, o que significa uma precisão de $1/16 = 0,0625$, ou seja, o primeiro candidato compatível encontrado estava na 16ª posição. Na recaptção de 50%, recuperam-se quatro compatíveis em um prefixo de 19 candidatos (precisão $\approx 0,2105$); em 100%, os oito compatíveis exigem um prefixo de 33 candidatos (precisão $\approx 0,2424$). Os onze pontos acima formam a curva de precisão e recaptção. Na agregação, em cada nível de recaptção, calcula-se a média aritmética das precisões dos 46 testes de referência, gerando a curva média de precisão e recaptção.

A seleção da melhor variante ocorre dentro de cada métrica. Para cada variante, calcula-se a média aritmética das onze precisões da curva agregada; permanece a configuração de maior média. A LCS e a LCCSS não possuem parâmetros e constituem variantes únicas. Quando duas configurações da mesma métrica produzem a mesma curva agregada (e.g., JPlag de sensibilidade 23 a 32), qualquer uma delas representa o desempenho da métrica; adota-se a de menor sensibilidade entre as empatadas.

Esse procedimento segue a perspectiva de recuperação da informação (Baeza-yates; Ribeiro-neto et al., 1999). Diz-se que um candidato foi recuperado quando seu grau de similaridade em relação ao teste de referência se encontra entre os mais elevados; a precisão é a fração de recuperados que a matriz de controle classifica como compatíveis, e a recaptção é a fração dos compatíveis desse teste que foram recuperados em cada um dos onze níveis padronizados.

A Tabela 3 apresenta a melhor variante de configuração selecionada para JPlag, Simian e Deckard. A LCS e a LCCSS não possuem parâmetros e, por isso, não aparecem na tabela.

Tabela 3 – Melhores variantes de configuração por métrica

Métrica	Parâmetro	Valor
JPlag	sensibilidade (-t)	23
Simian	limiar de linhas (-threshold)	8
	MIN_TOKENS	9
Deckard	STRIDE	11
	SIMILARITY	1,0

A Figura 17 apresenta uma linha para a melhor variante de cada métrica. O eixo horizontal percorre onze níveis padrão de recaptção, de 0% a 100% em incrementos de 10%, e o eixo vertical registra a precisão média correspondente, considerando todos os testes da suíte.

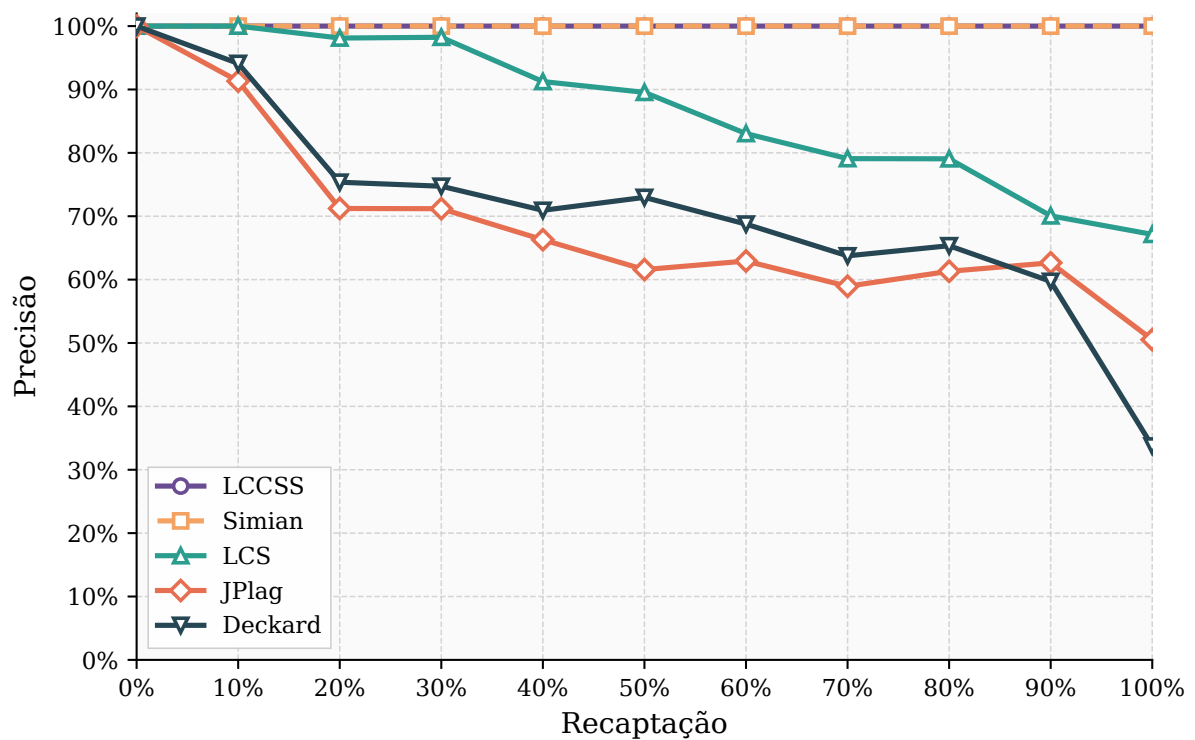


Figura 17 – Curvas médias de precisão e recaptação para a melhor variante de cada métrica

Fonte: Elaborado pelo autor.

Como se observa na Figura 17, todas as métricas alcançaram precisão superior a 0,9 no nível de recaptação de 10%. Entretanto, JPlag, Deckard e LCS apresentaram queda de precisão à medida que a recaptação aumenta, o que indica crescimento de falsos positivos quando se exige recuperar proporções maiores dos pares compatíveis. Em contraste, LCCSS e Simian mantiveram precisão 1,0 em todos os onze níveis de recaptação: todos os pares classificados como compatíveis foram recuperados sem falsos positivos nas ordenações produzidas por essas duas métricas. Por apresentarem os mesmos valores, as curvas dessas duas métricas ficam sobrepostas no gráfico.

A Tabela 4 resume os graus médios de similaridade atribuídos por cada métrica na melhor variante. Simian e LCCSS apresentam as menores médias e os maiores desvios padrão, padrão coerente com a matriz de controle, na qual apenas 219 dos 1035 pares (cerca de 21%) foram considerados compatíveis: a maior parte dos pares recebe grau baixo e uma minoria recebe graus mais elevados. As demais métricas distribuíram graus de forma mais uniforme, o que dificulta separar pares compatíveis de incompatíveis e explica a queda de precisão observada nas curvas.

Tabela 4 – Graus médios de similaridade na melhor variante de cada métrica

Métrica	Média	Desvio padrão	Mínimo	Máximo
JPlag	0,73	0,18	0,00	1,00
Simian	0,13	0,25	0,00	0,90
Deckard	0,33	0,13	0,19	1,00
LCS	0,66	0,13	0,38	0,98
LCCSS	0,23	0,25	0,05	0,98

5.7 DISCUSSÃO DOS RESULTADOS

A análise qualitativa dos graus atribuídos por LCS, JPlag e Deckard revela um padrão comum de falsos positivos: trechos comuns situados fora do prefixo inicial das configurações de acessórios elevam o grau de similaridade mesmo quando a configuração implícita não pode extrair esse código com segurança, conforme ilustrado na Figura 15. A LCS, em particular, atribui graus elevados quando o final das configurações coincide, embora o início diverja. JPlag e Deckard, por sua vez, podem atribuir graus baixos a pares cujo prefixo inicial é idêntico e que foram classificados como compatíveis, aumentando falsos negativos e prejudicando a recaptção.

LCCSS e Simian obtiveram desempenho equivalente nas curvas de precisão e recaptção. A diferença relevante para o *pipeline* reside na calibração: Simian exige a escolha do limiar de linhas adequado ao projeto (no experimento, a oitava configuração testada produziu o melhor resultado), enquanto a LCCSS opera sem parâmetros e percorre as sequências de comandos em tempo linear até a primeira divergência. Assim, a LCCSS oferece critério de similaridade alinhado à restrição de prefixo contíguo da configuração implícita sem depender de ajuste por projeto.

5.8 AMEAÇAS À VALIDADE

As ameaças à validade são discutidas segundo a classificação de Wohlin et al. (2012).

Validade de constructo. A matriz de controle foi definida pelos próprios autores e reflete julgamento sobre a relevância do reuso obtido por refatoração manual. Para mitigar subjetividade, a classificação foi repetida e os conflitos reavaliados. A experiência dos autores em implementação e manutenção de testes contribui para fundamentar os critérios adotados, mas não elimina o viés inerente ao critério humano.

Validade interna. A fonte de dados reúne testes bem estruturados e livres dos *test smells* catalogados por Greiler, Deursen e Storey (2013a), o que reduz interpretações equivocadas durante a classificação. Na medição, consideraram-se apenas as fases de configuração, exercício e verificação do padrão de quatro fases de Meszaros (2007); a fase de finalização (*tear down*) não entrou no escopo porque os testes selecionados não exigem limpeza explícita do SUT após a verificação, e o objetivo do experimento limita-se a identificar candidatos à configuração implícita, não a refatorar a finalização.

Validade de conclusão. O Róza não é um sistema de recuperação da informação, mas atribui graus de similaridade derivados da matriz correspondente, o que justifica o uso de precisão e recaptção como *proxy*. A generalização das conclusões permanece condicionada ao desenho experimental e ao tamanho da amostra.

Validade externa. Os resultados restringem-se a testes JUnit em Java de um único sistema de domínio específico. Suítes com estruturas de controle nas configurações, outros *frameworks* de teste ou outras linguagens podem alterar o desempenho relativo das métricas. A replicação em projetos adicionais constitui passo necessário antes de estender as conclusões à avaliação do *pipeline* completo de refatoração.

6 CONCLUSÃO

A duplicação do código de configuração de acessórios prejudica a manutenibilidade dos testes, mas sua remoção por meio da configuração implícita exige mais do que identificar trechos repetidos. À medida que a suíte cresce, torna-se necessário decidir como redistribuir os métodos entre classes de teste para aumentar o reuso sem introduzir mais duplicação, elevar o esforço total de desenvolvimento e manutenção ou alterar o resultado observado dos testes. Diante desse problema, este trabalho trata a distribuição dos testes como uma tarefa global de redistribuição de métodos entre classes e propõe o uso combinado de métricas de similaridade e algoritmos de agrupamento para automatizar a refatoração.

As principais contribuições propostas consistem em uma arquitetura em *pipeline* e no Róza, um *framework* extensível que instancia essa arquitetura com foco na configuração implícita. A separação do processo em etapas de carregamento, análise sintática, decomposição, medição, agrupamento, refatoração e escrita permite combinar implementações distintas sem acoplar todo o fluxo a uma linguagem, uma métrica ou um algoritmo específicos. O estudo sistemático apresentado neste trabalho reforça a originalidade dessa contribuição: embora existam abordagens automáticas e semiautomáticas para diferentes formas de refatoração de testes, não foi identificada uma arquitetura reusável que trate a configuração implícita como um problema global sobre toda a suíte.

No estágio atual da pesquisa, o Róza implementa as quatro primeiras etapas do *pipeline* e produz a matriz de similaridade utilizada pelo agrupamento. A avaliação empírica dessa etapa comparou cinco métricas em 46 testes e 1.035 pares classificados manualmente. A LCCSS e o Simian mantiveram precisão igual a 1,0 em todos os níveis de recaptção avaliados. Entretanto, a LCCSS alcançou esse resultado sem exigir calibração e, por considerar somente o prefixo comum contíguo das configurações, representa diretamente o código que pode ser extraído para um método de configuração implícita. Esses resultados fornecem evidência favorável à etapa de medição, mas ainda não permitem confirmar a hipótese de pesquisa em sua totalidade, pois não avaliam a redistribuição global dos testes nem a redução de duplicação e de esforço após a refatoração completa.

6.1 PRÓXIMOS PASSOS

A continuação imediata do trabalho consiste em implementar no Róza as etapas de agrupamento, refatoração e escrita. Para isso, deverão ser investigados algoritmos de agrupamento e suas configurações, incluindo critérios de parada que determinem quando uma nova fusão deixa de produzir reuso suficiente para compensar a duplicação ou a fragmentação introduzida. Em seguida, a etapa de refatoração deverá reconstruir as classes, extrair os prefixos comuns para métodos de configuração implícita e preservar o comportamento observado dos testes; a etapa de escrita deverá persistir os arquivos produzidos pelo processo.

Com o *pipeline* completo, será necessário avaliar a abordagem de ponta a ponta. Essa

avaliação deverá medir a duplicação antes e depois da refatoração, comparar a distribuição global com alternativas restritas às classes de origem e operacionalizar o esforço de refatoração e de manutenção. Também deverá verificar se o código produzido compila e se os testes preservam seus resultados. Assim, será possível determinar em que condições a redução de duplicação compensa a reorganização das classes e, conseqüentemente, avaliar a hipótese de que a redistribuição global reduz a duplicação e o esforço sem alterar o comportamento observado.

A avaliação das métricas também deverá ser replicada em projetos adicionais, uma vez que os resultados atuais se restringem a testes JUnit escritos em Java e provenientes de um único sistema. A ampliação da diversidade de sistemas, domínios e níveis de teste permitirá examinar a generalização dos resultados. A extensibilidade da arquitetura também poderá ser explorada com outras métricas de similaridade e algoritmos de agrupamento.

Também se pretende estender o Róza com um algoritmo de refatoração por configuração delegada. Acredita-se que essa estratégia exigirá uma métrica de similaridade específica, capaz de identificar trechos contíguos comuns em qualquer posição da configuração de acessórios, pois, diferentemente da configuração implícita, a configuração delegada não se restringe ao prefixo compartilhado pelos testes de uma classe. A incorporação dessa métrica e desse algoritmo sem modificações nas demais etapas permitirá avaliar a flexibilidade da arquitetura para admitir diferentes níveis de fatoração do código: desde a extração de prefixos comuns para métodos de configuração implícita até a extração de trechos compartilhados por subconjuntos de testes para métodos auxiliares.

REFERÊNCIAS

- ALMEIDA, R.; NOGUEIRA, S.; SAMPAIO, A. Combining sequential feature test cases to generate sound tests for concurrent features. **Science of Computer Programming**, v. 250, p. 103414, 2026.
- ALVESTAD, K. **Domain Specific Languages for Executable Specifications**. Dissertação (Mestrado) — Institutt for datateknikk og informasjonsvitenskap, 2007.
- BAEZA-YATES, R.; RIBEIRO-NETO, B. et al. **Modern information retrieval**. [S.l.]: ACM press New York, 1999. v. 463.
- BAXTER, I. D. et al. Clone detection using abstract syntax trees. In: **Proceedings International Conference on Software Maintenance**. [S.l.: s.n.], 1998. p. 368–377.
- BEIZER, B. **Software Testing Techniques**. 2. ed. [S.l.]: Van Nostrand Reinhold Company, 1990.
- BERNARD, E. et al. Tool support for refactoring manual tests. In: **2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)**. [S.l.: s.n.], 2020. p. 332–342.
- BERNER, S.; WEBER, R.; KELLER, R. K. Observations and lessons learned from automated testing. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.]: ACM, 2005. p. 571–579.
- BERTOLINO, A. Software testing research: Achievements, challenges, dreams. In: **2007 Future of Software Engineering**. [S.l.]: IEEE Computer Society, 2007. p. 85–103.
- CAO, Q.; YING, Y.; LI, P. Similarity metric learning for face recognition. In: **Proceedings of the IEEE International Conference on Computer Vision**. [S.l.: s.n.], 2013. p. 2408–2415.
- CHRISTENSEN, C. A. et al. A unit-test framework for database applications. In: **2006 10th International Database Engineering and Applications Symposium (IDEAS'06)**. [S.l.]: IEEE, 2006. p. 11–20.
- DEREZIŃSKA, A.; SOBIERAJ, O. Automated unit test refactoring to eliminate test smells and support test maintenance. In: **Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering**. [S.l.]: SCITEPRESS, 2026. p. 1097–1104.
- DEURSEN, A. van et al. Refactoring test code. In: **Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)**. [S.l.: s.n.], 2001. p. 92–95.
- DEZA, M. M.; DEZA, E. **Encyclopedia of Distances**. [S.l.]: Springer, 2009.
- DICE, L. R. Measures of the amount of ecologic association between species. **Ecology**, v. 26, n. 3, p. 297–302, 1945.
- DONALDSON, J. L.; LANCASTER, A.-M.; SPOSATO, P. H. A plagiarism detection system. **SIGCSE Bull.**, v. 13, n. 1, p. 21–25, 1981.

- DORNELES, C. F. et al. Measuring similarity between collection of values. In: **Proceedings of the 6th annual ACM international workshop on Web information and data management**. [S.l.: s.n.], 2004. p. 56–63.
- DUCASSE, S.; RIEGER, M.; DEMEYER, S. A language independent approach for detecting duplicated code. In: **Proceedings IEEE International Conference on Software Maintenance**. [S.l.: s.n.], 1999. p. 109–118.
- EMERY, D. H. Writing maintainable automated acceptance tests. In: **Agile Testing Workshop, Agile Development Practices**. Orlando, Florida: [s.n.], 2009.
- ERFANI, M.; KEIVANLOO, I.; RILLING, J. Opportunities for clone detection in test case recommendation. In: **2013 IEEE 37th Annual Computer Software and Applications Conference Workshops**. [S.l.: IEEE, 2013. p. 65–70.
- ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: **Proceedings of the Second International Conference on Knowledge Discovery and Data Mining**. [S.l.: s.n.], 1996. p. 226–231.
- FERRANTE, J.; OTTENSTEIN, K. J.; WARREN, J. D. The program dependence graph and its use in optimization. **ACM Transactions on Programming Languages and Systems**, v. 9, n. 3, p. 319–349, 1987.
- FOWLER, M. **Refactoring: Improving the Design of Existing Code**. [S.l.]: Addison-Wesley, 1999.
- FREEMAN, S.; PRYCE, N. **Growing Object-Oriented Software, Guided by Tests**. [S.l.]: Pearson Education, 2009.
- FREITAS, W. d. V. C. et al. Reusable testing actions on test case automation for mobile devices. In: **Proceedings of the XXIII Brazilian Symposium on Software Quality**. [S.l.]: ACM, 2024. p. 460–468.
- GAO, Y. et al. Automated unit test refactoring. **Proceedings of the ACM on Software Engineering**, v. 2, n. FSE, p. 713–733, 2025.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Automated detection of test fixture strategies and smells. In: **2013 IEEE Sixth International Conference on Software Testing, Verification and Validation**. [S.l.]: IEEE, 2013. p. 322–331.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Strategies for avoiding text fixture smells during software evolution. In: **Proceedings of the 10th Working Conference on Mining Software Repositories**. [S.l.]: IEEE Press, 2013. p. 387–396.
- GUO, Z. et al. Pc-trt: A test case reuse and generation tool to achieve high path coverage for unit test. **SoftwareX**, v. 28, p. 101918, 2024.
- HAMMING, R. W. Error detecting and error correcting codes. **The Bell System Technical Journal**, v. 29, n. 2, p. 147–160, 1950.
- HARROLD, M. J.; GUPTA, R.; SOFFA, M. L. A methodology for controlling the size of a test suite. **ACM Transactions on Software Engineering and Methodology**, v. 2, n. 3, p. 270–285, 1993.

HARTIGAN, J. A.; WONG, M. A. Algorithm as 136: A k-means clustering algorithm. **Journal of the Royal Statistical Society Series C**, v. 28, n. 1, p. 100–108, 1979.

HAUGSET, B.; HANSSEN, G. K. Automated acceptance testing: A literature review and an industrial case study. In: **Agile 2008 Conference**. [S.l.]: IEEE, 2008. p. 27–38.

HAUPTMANN, B. et al. Generating refactoring proposals to remove clones from automated system tests. In: **2015 IEEE 23rd International Conference on Program Comprehension**. [S.l.: s.n.], 2015. p. 115–124.

HEMMATI, H.; ARCURI, A.; BRIAND, L. Reducing the cost of model-based testing through test case diversity. In: **Testing Software and Systems**. [S.l.]: Springer, 2010. p. 63–78.

HEMMATI, H.; BRIAND, L. An industrial investigation of similarity measures for model-based test case selection. In: **Proceedings of the 21st IEEE International Symposium on Software Reliability Engineering**. [S.l.]: IEEE, 2010. p. 141–150.

HEMMATI, H. et al. An enhanced test case selection approach for model-based testing: An industrial case study. In: **Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering**. [S.l.]: ACM, 2010. p. 267–276.

HEREDIA-BRAVO, J. M. et al. Refactoring process as part of adapting test tools to real software. In: **Applied Computer Science**. [S.l.]: Springer, 2026. p. 16–26.

HIRSCHBERG, D. S. Algorithms for the longest common subsequence problem. **Journal of the ACM**, v. 24, n. 4, p. 664–675, 1977.

HOLMES, R.; MURPHY, G. C. Using structural context to recommend source code examples. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.: s.n.], 2005. p. 117–125.

JACCARD, P. The distribution of the flora in the alpine zone. **New Phytologist**, v. 11, n. 2, p. 37–50, 1912.

JAIN, A. K.; MURTY, M. N.; FLYNN, P. J. Data clustering: a review. **ACM Computing Surveys**, v. 31, n. 3, p. 264–323, 1999.

JIANG, L. et al. Deckard: Scalable and accurate tree-based detection of code clones. In: **29th International Conference on Software Engineering (ICSE'07)**. [S.l.: s.n.], 2007. p. 96–105.

JOHNSON, R. A.; WICHERN, D. W. **Applied multivariate statistical analysis**. Upper Saddle River, NJ: Prentice Hall, 2002.

KAMIYA, T.; KUSUMOTO, S.; INOUE, K. Ccfinder: a multilinguistic token-based code clone detection system for large scale source code. **IEEE Transactions on Software Engineering**, v. 28, n. 7, p. 654–670, 2002.

KIRKPATRICK, S.; JR, C. D. G.; VECCHI, M. P. Optimization by simulated annealing. **Science**, v. 220, n. 4598, p. 671–680, 1983.

KOHONEN, T. Essentials of the self-organizing map. **Neural Networks**, v. 37, p. 52–65, 2013.

KOMONDOOR, R.; HORWITZ, S. Using slicing to identify duplication in source code. In: **International Static Analysis Symposium**. [S.l.]: Springer, 2001. p. 40–56.

KRINKE, J. Identifying similar code with program dependence graphs. In: **Proceedings Eighth Working Conference on Reverse Engineering**. [S.l.: s.n.], 2001. p. 301–309.

LAMBIASE, S. et al. Just-in-time test smell detection and refactoring: The darts project. In: **Proceedings of the 28th International Conference on Program Comprehension**. [S.l.: s.n.], 2020. p. 441–445.

LEDRU, Y. et al. Prioritizing test cases with string distances. **Automated Software Engineering**, v. 19, n. 1, p. 65–95, 2012.

LEVENSHTAIN, V. I. Binary codes capable of correcting deletions, insertions, and reversals. **Soviet Physics Doklady**, v. 10, n. 8, p. 707–710, 1966.

LONGO, D. H. et al. Fixture setup through object notation for implicit test fixtures. **Journal of Computer Science**, v. 11, n. 6, p. 794, 2015.

LUXBURG, U. von. A tutorial on spectral clustering. **Statistics and Computing**, v. 17, n. 4, p. 395–416, 2007.

MAIER, F.; FELDERER, M. Detection of test smells with basic language analysis methods and its evaluation. In: **2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)**. [S.l.: s.n.], 2023. p. 897–904.

MESZAROS, G. **xUnit test patterns: Refactoring test code**. [S.l.]: Pearson Education, 2007.

MESZAROS, G.; SMITH, S. M.; ANDREA, J. The test automation manifesto. In: **Extreme Programming and Agile Methods - XP/Agile Universe 2003**. [S.l.]: Springer, 2003. p. 73–81.

MIRANDA, B. et al. Fast approaches to scalable similarity-based test case prioritization. In: **Proceedings of the 40th International Conference on Software Engineering**. [S.l.]: ACM, 2018. p. 222–232.

MURUGESAN, S. Attitude towards testing: a key contributor to software quality. In: **Proceedings of 1994 1st International Conference on Software Testing, Reliability and Quality Assurance (STRQA'94)**. [S.l.: s.n.], 1994. p. 111–115.

NARANG, M.; DU, H.; JONES, J. A. What's dat smell? untangling and weaving the disjoint assertion tangle test smell. In: **2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)**. [S.l.]: IEEE, 2025. p. 1260–1272.

NG, R. T.; HAN, J. Clarans: a method for clustering objects for spatial data mining. **IEEE Transactions on Knowledge and Data Engineering**, v. 14, n. 5, p. 1003–1016, 2002.

OTTENSTEIN, K. J. An algorithmic approach to the detection and prevention of plagiarism. **SIGCSE Bull.**, v. 8, n. 4, p. 30–41, 1976.

PERUMA, A. et al. tsdetect: an open source test smells detection tool. In: **Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering**. [S.l.: s.n.], 2020. p. 1650–1654.

PIZZINI, A.; REINEHR, S.; MALUCELLI, A. Automatic refactoring method to remove eager test smell. In: **Proceedings of the XXI Brazilian Symposium on Software Quality**. [S.l.: s.n.], 2023.

- PIZZINI, A.; REINEHR, S.; MALUCELLI, A. Sentinel: A process for automatic removing of test smells. In: **Proceedings of the XXII Brazilian Symposium on Software Quality**. [S.l.: s.n.], 2023. p. 80–89.
- PRECHELT, L.; MALPOHL, G.; PHILIPPSEN, M. Finding plagiarisms among a set of programs with jplag. **J. Univers. Comput. Sci.**, v. 8, n. 11, p. 1016, 2002.
- QUSEF, A. et al. Post-refactoring recovery of unit tests: An automated approach. **International Journal of Interactive Mobile Technologies**, v. 17, n. 8, 2023.
- RAGKHITWETSAGUL, C.; KRINKE, J.; CLARK, D. A comparison of code similarity analysers. **Empirical Software Engineering**, v. 23, n. 4, p. 2464–2519, 2018.
- ROKACH, L.; MAIMON, O. Clustering methods. In: **Data mining and knowledge discovery handbook**. [S.l.]: Springer, 2005. p. 321–352.
- ROY, C. K.; CORDY, J. R. Nicad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In: **2008 16th IEEE International Conference on Program Comprehension**. [S.l.: s.n.], 2008. p. 172–181.
- SAHAMI, M.; HEILMAN, T. D. A web-based kernel function for measuring the similarity of short text snippets. In: **Proceedings of the 15th international conference on World Wide Web**. [S.l.: s.n.], 2006. p. 377–386.
- SANTANA, R. et al. **Refactoring Assertion Roulette and Duplicate Assert test smells: a controlled experiment**. 2022. ArXiv:2207.05539. Disponível em: <https://arxiv.org/abs/2207.05539>.
- SHAMSHIRI, S. et al. How do automatically generated unit tests influence software maintenance? In: **2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)**. [S.l.: s.n.], 2018. p. 250–261.
- SHI, Y.-Q.; HUANG, S.; WAN, J.-Y. A reuse-oriented clustering method for test cases. In: **2022 9th International Conference on Dependable Systems and Their Applications (DSA)**. [S.l.: s.n.], 2022. p. 153–162.
- SIBBALD, D. B.; WENTZELL, P. D.; WADE, A. P. Display methods for dendrograms. **TrAC Trends in Analytical Chemistry**, v. 8, n. 8, p. 289–291, 1989. ISSN 0165-9936. Disponível em: <https://www.sciencedirect.com/science/article/pii/0165993689850629>.
- SILVA, L. P.; VILAIN, P. Reuse of fixture setup between test classes. In: **SEKE**. [S.l.: s.n.], 2017. p. 224–229.
- SILVA, L. P. d.; VILAIN, P. Lccss: A similarity metric for identifying similar test code. In: **Proceedings of the 14th Brazilian Symposium on Software Components, Architectures, and Reuse**. New York, NY, USA: Association for Computing Machinery, 2020. p. 91–100. Disponível em: <https://doi.org/10.1145/3425269.3425283>.
- SOARES, E.; RIBEIRO, M.; SANTOS, A. A multimethod study of test smells: Cataloging removal and new types. In: **Proceedings of the XXIII Brazilian Symposium on Software Quality**. [S.l.]: ACM, 2024. p. 676–686.
- TEIXEIRA, T. S. R.; SILVEIRA, F. F.; GUERRA, E. M. Mutation testing in test code refactoring: Leveraging mutants to ensure behavioral consistency. In: **Agile Processes in Software Engineering and Extreme Programming**. [S.l.]: Springer, 2025. p. 176–185.

TIWARI, R.; GOEL, N. Reuse: reducing test effort. **ACM SIGSOFT Software Engineering Notes**, v. 38, n. 2, p. 1–11, 2013.

VELOSO, V. G. When testing meets refactoring: Catalogue, detection, and recommendation. In: **Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering**. [S.l.]: ACM, 2026. p. 35–38.

VELOSO, V. G.; TSANTALIS, N.; CHEN, T.-H. P. A mixed-method in-depth study of test-specific refactorings. **Empirical Software Engineering**, v. 32, n. 1, 2026.

WANG, G. et al. Test architecture generation by leveraging bert and control and data flows. In: **Engineering of Complex Computer Systems**. [S.l.]: Springer, 2024. p. 125–145.

WANG, W.; YANG, J.; MUNTZ, R. Sting: A statistical information grid approach to spatial data mining. In: **Proceedings of the 23rd International Conference on Very Large Data Bases**. [S.l.: s.n.], 1997. p. 186–195.

WANG, X. et al. An automatic refactoring framework for replacing test-production inheritance by mocking mechanism. In: **Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering**. [S.l.: s.n.], 2021. p. 540–552.

WANG, X. et al. Jmocker: refactoring test-production inheritance by mockito. In: **Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings**. [S.l.: s.n.], 2022. p. 125–129.

WANG, X. et al. From inheritance to mockito: An automatic refactoring approach. **IEEE Transactions on Software Engineering**, v. 49, n. 4, p. 2791–2814, 2023.

WASHIZAKI, H. **Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0**. IEEE Computer Society, 2024. Disponível em: <https://www.swebok.org/>.

WOHLIN, C. et al. **Experimentation in Software Engineering**. [S.l.]: Springer Science & Business Media, 2012.

WONG, W. E. et al. A study of effective regression testing in practice. In: **Proceedings of the Eighth International Symposium on Software Reliability Engineering**. [S.l.]: IEEE, 1997. p. 264–274.

XUAN, J. et al. B-refactoring: Automatic test code refactoring to improve dynamic analysis. **Information and Software Technology**, v. 76, p. 65–80, 2016.

YANG, Y. et al. Testqual: Conceptualizing software testing as a service. **e-Service Journal**, Indiana University Press, v. 7, n. 2, p. 46–65, 2011. ISSN 15288226, 15288234. Disponível em: <http://www.jstor.org/stable/10.2979/eservicej.7.2.46>.

ZHAO, G. Automated detection and refactoring of mock clones in java projects. In: **2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)**. [S.l.]: IEEE, 2025. p. 112–116.

ZHAO, G. et al. Mock clones in the wild: An empirical investigation across six open-source projects. In: **2025 32nd Asia-Pacific Software Engineering Conference (APSEC)**. [S.l.]: IEEE, 2025. p. 395–406.

APÊNDICE A – VIOLAÇÕES DE CÓDIGO DE TESTE REGISTRADAS NA ANÁLISE SINTÁTICA

Este apêndice detalha as violações mencionadas no Capítulo 4. Cada subseção apresenta a condição que caracteriza uma violação, o motivo de sua adoção e um exemplo.

A.1 VIOLAÇÕES DE CLASSE

Uma violação de classe é registrada quando o elemento identificado afeta a classe como um todo, o arquivo que a declara ou algum elemento compartilhado por seus testes. Nesse caso, todos os métodos de teste da classe são excluídos das etapas seguintes do *pipeline*.

A.1.1 Importação com curinga

Ocorre quando o arquivo declara uma importação com curinga. Essa violação foi adotada porque a transformação pode impedir a compilação ou mudar o significado dos nomes usados pelos testes. O Róza analisa a sintaxe, mas não resolve cada nome para o tipo ao qual ele se refere. No trecho, o uso de `List` não registra junto ao nome que o tipo foi obtido de `java.util.*`. Se o teste for movido sem esse curinga, a classe gerada não compila. Se todos os curingas das diferentes classes de origem forem copiados, dois pacotes podem oferecer tipos com o mesmo nome e tornar a referência ambígua. Essa cópia também pode introduzir importações não utilizadas na classe gerada. Uma importação explícita identifica o tipo pretendido e permite qualificá-lo quando necessário; o curinga não oferece essa informação ao Róza. Por isso, o recurso é recusado para evitar que a refatoração produza testes que não compilem ou passem a referenciar outro tipo.

```
import java.util.*;
List contas;
```

A.1.2 Mais de uma classe de nível superior no mesmo arquivo

Ocorre quando o arquivo declara mais de uma classe de nível superior. Embora Java permita essa construção, este trabalho a considera uma má prática, pois o arquivo deixa de corresponder a uma única unidade de código. No trecho, `TesteConta` e `TesteBanco` possuem testes e configurações independentes. Interpretar cada classe separadamente e gerar um arquivo para cada uma seria possível, mas exigiria que o carregamento deixasse de tratar cada arquivo como uma única classe de teste. Esse suporte foi excluído tanto para manter a correspondência entre arquivo e classe quanto para simplificar a implementação do Róza.

```
class TesteConta {
    Conta conta;
    @Before void criaConta() { conta = new Conta(); }
```

```

    @Test void deposita() { conta.depositar(10); }
}
class TesteBanco {
    Banco banco;
    @Before void criaBanco() { banco = new Banco(); }
    @Test void abre() { banco.abrir(); }
}

```

A.1.3 Classe aninhada

Ocorre quando uma classe de teste declara outra classe em seu interior. Essa violação foi adotada para simplificar a implementação do Róza. Seu modelo representa cada classe de teste por um único conjunto de atributos, métodos de configuração e métodos de teste, sem registrar relações entre classes externas e internas. No trecho, antes de executar cadastra, o JUnit 5 executa primeiro `criaBanco`, da classe externa, e depois `criaConta`, da classe interna. O suporte seria possível, mas exigiria representar os diferentes níveis da classe aninhada e combinar suas configurações na ordem determinada pelo JUnit. Essa extensão não integra o escopo deste trabalho.

```

class TesteBanco {
    Banco banco;
    @BeforeEach void criaBanco() { banco = new Banco(); }
    @Nested class Cadastro {
        Conta conta;
        @BeforeEach void criaConta() { conta = new Conta(); }
        @Test void cadastra() { banco.cadastrar(conta); }
    }
}

```

A.1.4 Record aninhado

Ocorre quando a classe de teste declara um record em seu interior. Essa violação foi adotada pelas mesmas razões que recusam a classe aninhada e o enumerador: o modelo do Róza representa cada classe de teste por um único conjunto de atributos, métodos de configuração e métodos de teste, sem tratar declarações adicionais de tipos como unidades independentes de redistribuição. No trecho, `Par` é um tipo usado por `cria`. Mover o teste sem copiar o record impede a compilação; copiá-lo para todas as classes geradas pode introduzir tipos que alguns testes não utilizam. Sem analisar quais testes dependem de cada record, não há uma transformação definida para o subconjunto tratado. Por isso, a classe deixa de integrar as etapas seguintes, ainda que o analisador conserve o texto do tipo aninhado para inspeção e para a opção de ignorar violações.

```

class TestePar {
    private record Par(int a, int b) {}
    @Test void cria() {
        Par par = new Par(1, 2);
    }
}

```

```

        assertEquals(1, par.a());
    }
}

```

A.1.5 Herança

Ocorre quando a classe de teste estende outra classe. Essa violação foi adotada para simplificar a implementação, evitar que uma configuração herdada seja omitida durante a refatoração e desencorajar o uso de herança em testes. Nesse contexto, a herança constitui uma má prática porque oculta dependências de configuração e acopla os testes à hierarquia de classes. O modelo do Róza não registra a superclasse nem associa os elementos herdados aos testes da subclasse. No trecho, `TesteBanco` herda o campo `banco` e o método `criaBanco` de `TesteBaseBanco`. Antes de executar `abre`, o JUnit executa o `@Before` herdado. O suporte seria possível, mas exigiria localizar as superclasses, percorrer toda a hierarquia e incorporar seus atributos e métodos de configuração na ordem correta. Sem esse tratamento, o teste refatorado poderia perder a configuração herdada e mudar de comportamento.

```

// TesteBaseBanco.java
class TesteBaseBanco {
    Banco banco;
    @Before void criaBanco() { banco = new Banco(); }
}
// TesteBanco.java
class TesteBanco extends TesteBaseBanco {
    @Test void abre() { banco.abrir(); }
}

```

A.1.6 Classe abstrata

Ocorre quando a classe de teste é declarada como abstrata. Uma classe abstrata pode servir de base para subclasses, mas não pode ser instanciada diretamente pelo JUnit. Como o Róza não trata herança e redistribui testes entre classes concretas, uma classe abstrata isolada não constitui uma unidade de teste com significado dentro do modelo adotado. Por isso, ela não pode ser uma classe de origem nem o destino gerado pela refatoração.

```

abstract class TesteBanco {
    @Test void abre() {}
}

```

A.1.7 Classe genérica

Ocorre quando a classe de teste declara parâmetros de tipo. Essa construção não possui significado no modelo adotado, que redistribui métodos entre classes concretas. No trecho, `T` não identifica um tipo concreto para `valor`. Além disso, testes provenientes de classes distin-

tas podem declarar parâmetros com nomes ou limites diferentes, sem que exista uma escolha única para a classe resultante. Como o Róza não instancia combinações de tipos nem preserva vínculos genéricos entre classes de origem, não há uma transformação definida para esse caso.

```
class TesteRepositorio<T> {
    Repositorio<T> repositorio;
    T valor;
    @Test void armazena() { repositorio.salvar(valor); }
}
```

A.1.8 Anotação na classe

Ocorre quando a classe de teste possui uma anotação. Essa violação foi adotada porque a anotação pode alterar a descoberta, a instanciação ou o ciclo de vida dos testes, e ignorá-la durante a redistribuição pode mudar o comportamento observado. No trecho, `@TestInstance` configura um ciclo de vida por classe, no qual o JUnit 5 usa a mesma instância de `TesteBanco` em todos os métodos, em vez de criar uma para cada teste. Copiar a anotação para todas as classes geradas ou removê-la pode, respectivamente, ampliar ou eliminar o compartilhamento de estado. Como o Róza não interpreta a semântica das anotações de classe, recusa o caso em vez de produzir uma refatoração potencialmente incorreta.

```
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class TesteBanco {
    Banco banco;
    @BeforeEach void criaBanco() { banco = new Banco(); }
    @Test void abre() { banco.abrir(); }
}
```

A.1.9 Inicializador de classe

Ocorre quando a classe declara um inicializador estático. Essa construção introduz estado ou efeitos compartilhados por todos os testes e, por isso, é tratada como uma má prática que pode comprometer sua independência. No trecho, `Banco.abrir()` executa uma única vez, quando a classe é carregada, e não antes de cada teste. Redistribuir os métodos entre novas classes pode fazer o bloco executar mais vezes ou em outro momento. A violação evita que a refatoração altere esse efeito global e, consequentemente, o resultado dos testes.

```
class TesteBanco {
    static { Banco.abrir(); }
    @Test void consulta() {}
}
```

A.1.10 Construtor explícito

Ocorre quando a classe declara um construtor explícito. O Róza considera como configuração implícita apenas os comandos de `@Before` ou `@BeforeEach`. O uso do construtor para preparar testes fica fora desse contrato, e seu comportamento pode depender da versão do JUnit e de extensões que controlam a criação ou a injeção de instâncias. No trecho, `new Banco()` executa durante a construção do objeto, antes do método de configuração implícita. Como a ordem e os efeitos que deveriam ser preservados não estão suficientemente definidos no modelo do Róza, o construtor é recusado para evitar que a refatoração omita ou reposicione essa preparação.

```
class TesteBanco {
    Banco banco;
    public TesteBanco() { banco = new Banco(); }
    @Test void abre() { banco.abrir(); }
}
```

A.1.11 Enumerador

Ocorre quando o arquivo contém um enumerador. O enumerador pode ser um tipo auxiliar utilizado pelos testes, mas o modelo do Róza preserva apenas os elementos da classe de teste e não representa declarações adicionais de tipos. Para suportá-lo, seria necessário identificar quais testes dependem do enumerador e decidir em quais arquivos gerados ele deveria permanecer ou ser copiado. Esse tratamento foi excluído para simplificar a implementação.

```
class TesteConta {
    enum Status { ATIVO, INATIVO }
    @Test void ativa() {
        Status status = Status.ATIVO;
        assertEquals(Status.ATIVO, status);
    }
}
```

A.1.12 Atributo estático

Ocorre quando a classe declara um atributo estático. O atributo compartilha o mesmo valor entre todas as instâncias da classe, o que constitui uma má prática quando os testes o alteram e passam a depender da ordem de execução. No trecho, `a` abre o banco e `b` fecha a mesma instância. Após a redistribuição, copiar o atributo para várias classes separaria um estado antes compartilhado, enquanto mantê-lo em uma única classe preservaria um acoplamento entre testes que foram movidos. Como ambas as opções podem mudar o resultado observado, o Róza recusa a classe.

```
static Banco banco;
@Test void a() { banco.abrir(); }
```

```
@Test void b() { banco.fechar(); }
```

A.1.13 Atributo anotado

Ocorre quando um atributo possui uma anotação. Essa violação foi adotada porque *frameworks* e extensões podem usar a anotação para criar, substituir ou destruir o valor do atributo fora do método de configuração implícita. No trecho, @TempDir solicita ao JUnit que forneça um diretório temporário antes do teste. Mover o atributo ou o método sem compreender essa anotação pode eliminar a injeção ou alterar seu ciclo de vida. Como o Róza não interpreta anotações de atributos, recusa o caso para evitar a quebra dos testes refatorados.

```
@TempDir Path pasta;
```

A.1.14 Atributo inicializado no campo

Ocorre quando o atributo é inicializado na própria declaração. Essa violação simplifica a implementação ao concentrar a configuração tratada pelo Róza em @Before ou @BeforeEach. No trecho, new Banco() executa durante a criação da instância e antes do método de configuração implícita. Seria possível incorporar a inicialização ao teste decomposto, mas isso exigiria preservar sua posição em relação ao construtor e aos demais inicializadores. Sem esse tratamento, movê-la para outro ponto poderia alterar a ordem da configuração e o comportamento do teste.

```
Banco banco = new Banco();
```

A.1.15 Método auxiliar

Ocorre quando a classe declara um método que não é de teste nem de configuração implícita. Essa violação foi adotada para simplificar a implementação, pois tratar o método exigiria analisar chamadas entre métodos e determinar quais comandos do auxiliar pertencem à configuração de cada teste. No trecho, deposita chama criarConta, mas a criação da conta está no corpo do método auxiliar. Copiar somente a chamada para uma nova classe produziria uma referência ausente; copiar ou incorporar o auxiliar sem analisar seus usos poderia duplicar efeitos ou mudar seu escopo. Por isso, o Róza recusa a classe para não gerar testes incompletos.

```
void criarConta() { conta = new Conta(); }
@Test void deposita() {
    criarConta();
    conta.depositar(10);
}
```


A.1.16 Método de finalização ou demais métodos de ciclo de vida

Ocorre quando a classe declara um método de finalização ou outro método de ciclo de vida distinto da configuração implícita. São registradas as anotações `@After`, `@AfterEach`, `@AfterClass`, `@BeforeClass`, `@BeforeAll` e `@AfterAll`. O suporte foi excluído para simplificar a implementação, que considera apenas a configuração executada antes de cada teste. No trecho, `limpar` executa depois do teste. Se os métodos forem redistribuídos sem que essa finalização acompanhe cada grupo, recursos podem permanecer abertos; se ela for copiada indiscriminadamente, um recurso compartilhado pode ser encerrado mais de uma vez. A violação evita que a refatoração altere o ciclo de vida e quebre os testes.

```
@After public void limpar() { banco.fechar(); }
```

A.1.17 Mais de um método de configuração implícita

Ocorre quando a classe declara mais de um método de configuração implícita. O Róza representa a configuração compartilhada como uma única sequência e, para simplificar a implementação, não combina vários métodos `@Before` ou `@BeforeEach`. Além disso, a ordem entre esses métodos não deve ser inferida pela posição textual de suas declarações. No trecho, executar `a` antes de `b` pode produzir um estado diferente de executar `b` antes de `a`. Como o comportamento que a classe gerada deveria preservar não está suficientemente definido pelo código analisado, a violação evita uma mudança na ordem da configuração.

```
@Before void a() { conta = new Conta(); }
@Before void b() { banco = new Banco(); }
```

A.1.18 Método de configuração implícita fora do contrato

Ocorre quando o método de configuração implícita é estático, possui parâmetros ou tem tipo de retorno distinto de `void`. Essas formas não constituem a configuração implícita convencional adotada pelo Róza. No JUnit 4, elas não atendem ao contrato de `@Before`; no JUnit 5, alguns parâmetros podem ser fornecidos por extensões, cuja semântica o Róza não interpreta. No trecho, o método estático pertence à classe, e não à instância criada para cada teste. Como não há uma transformação com significado dentro do modelo adotado e o suporte a extensões foi excluído para simplificar a implementação, a classe é recusada.

```
@Before static void criaBanco() { banco = new Banco(); }
```

A.2 VIOLAÇÕES DE MÉTODO

As violações desta seção aplicam-se a um método de teste. O analisador exclui apenas o método afetado, de modo que os demais testes da classe possam seguir no *pipeline*. Quando

a mesma construção ocorre no método de configuração implícita, o analisador registra uma violação de classe, pois a extração da configuração compartilhada ficaria comprometida para todos os testes.

A.2.1 Teste parametrizado ou com ciclo de vida distinto

Ocorre quando o método é parametrizado, repetido, teórico, gerado por fábrica ou definido por um modelo. São registradas as anotações `@ParameterizedTest`, `@RepeatedTest`, `@Theory`, `@TestFactory` e `@TestTemplate`. O suporte foi excluído para simplificar o Róza, cujo modelo representa cada método como um único teste sem parâmetros. No trecho, `aceita` é executado uma vez para cada valor fornecido. Tratar esse caso exigiria representar as invocações produzidas pelo *framework* e preservar suas fontes, nomes e ciclos de vida. Mover o método como se fosse um teste simples poderia eliminar essas invocações ou mudar os valores recebidos e, portanto, quebrar o teste refatorado.

```
@ParameterizedTest
@ValueSource(strings = {"ativa", "inativa"})
void aceita(String valor) {}
```

A.2.2 Anotação repetida no mesmo método

Ocorre quando a mesma anotação aparece mais de uma vez no método. Essa construção não possui significado no modelo adotado. Anotações como `@Test` não são repetíveis em Java, de modo que o trecho não compila e não corresponde a um método de teste executável. Como não há comportamento válido a preservar nem transformação aplicável, o Róza registra a violação.

```
@Test
@Test
public void abre() {}
```

A.2.3 Anotação distinta de teste ou de configuração implícita

Ocorre quando o método possui uma anotação diferente das anotações de teste e de configuração implícita admitidas pelo Róza. Essa violação foi adotada porque anotações adicionais podem mudar a descoberta, as condições ou o ciclo de vida do teste. No trecho, `@Disabled` impede a execução de `abre`. Como o Róza não interpreta a semântica dessas anotações, não pode determinar se basta copiá-las ou se elas dependem de outra anotação, extensão ou configuração da classe. Prosseguir poderia fazer um teste ignorado passar a executar ou produzir outra mudança de comportamento após a refatoração.

```
@Disabled
@Test
public void abre() {}
```

A.2.4 Atributo nas anotações de teste ou de configuração implícita

Ocorre quando as anotações `@Test`, `@Before` ou `@BeforeEach` recebem atributos. Essa violação evita que a refatoração mude o critério de sucesso ou a forma de execução do teste. No trecho, `expected` faz abre passar quando seu corpo lança `Exception`. Se um comando que lança essa exceção for extraído para `@Before`, ela pode ocorrer fora do ponto ao qual o atributo se aplica. Remover, modificar ou copiar o atributo sem considerar essa relação pode transformar um teste aprovado em reprovado, ou o contrário.

```
@Test(expected = Exception.class)
public void abre() { banco.abrir(); }
```

A.2.5 Método de teste fora do contrato

Ocorre quando o método de teste é privado, estático, sem corpo, possui parâmetros ou tem tipo de retorno distinto de `void`. Essas formas ficam fora do contrato simplificado do Róza, que representa testes como métodos de instância, com corpo, sem parâmetros e sem retorno. Algumas delas não constituem testes válidos na versão correspondente do JUnit; outras dependem de extensões que fornecem parâmetros ou alteram a execução. No trecho, `private` impede que o método seja descoberto como um teste convencional. Como o comportamento esperado não está definido pelo modelo adotado, não há uma redistribuição com significado sem interpretar as regras adicionais do *framework*.

```
@Test private void abre() {}
```

A.2.6 Laço

Ocorre na presença de `for`, `for-each`, `while` ou `do-while`. O suporte foi excluído para simplificar a implementação, que compara a configuração como uma sequência linear de comandos. No trecho, a quantidade de execuções de `contas.add` depende de `n`, e o corpo pode ainda alterar variáveis usadas na condição ou depois do laço. Seria possível mover o laço inteiro, mas isso exigiria analisar seu escopo, suas dependências e seu fluxo interno. Sem essa análise, extrair apenas partes coincidentes poderia mudar a quantidade ou a ordem dos efeitos e quebrar o teste.

```
for (int i = 0; i < n; i++) {
    contas.add(new Conta());
}
```

A.2.7 Comando try

Ocorre na presença de um bloco try. Essa violação simplifica a análise ao excluir fluxos de exceção da sequência linear tratada pelo Róza. No trecho, se `banco.abrir()` lançar uma exceção, a execução segue para `catch`; caso contrário, esse bloco não é executado. Seria possível mover o try completo, mas extrair comandos de seu interior ou combiná-lo com trechos de outros testes exigiria preservar os blocos `catch` e `finally`, bem como o ponto em que cada exceção é tratada. Uma transformação incompleta poderia mudar o caminho executado e o resultado do teste.

```
try {
    banco.abrir();
} catch (Exception execao) {}
```

A.2.8 Comandos switch, break, continue ou throw explícito

Ocorre na presença de `switch`, `break`, `continue` ou `throw` explícito. O suporte foi excluído para manter a configuração como uma sequência linear, sem ramificações ou saltos. No trecho, `throw` encerra o fluxo normal e impede a execução dos comandos seguintes. Os demais elementos podem selecionar outro ramo, sair de um bloco ou iniciar a próxima iteração. Tratar essas construções exigiria preservar o contexto de controle ao qual pertencem; movê-las isoladamente pode até produzir código inválido ou alterar o caminho executado. A violação evita que a refatoração quebre a compilação ou o comportamento do teste.

```
throw new IllegalStateException();
```

A.2.9 Bloco sincronizado ou comando rotulado

Ocorre na presença de um bloco `synchronized` ou de um comando rotulado. Essas construções foram excluídas para simplificar a implementação e evitar alterações em relações que dependem do contexto original. No trecho, `synchronized (this)` utiliza a instância da classe como cadeado. Depois de mover o teste, `this` passa a designar outra instância e pode deixar de sincronizar com o mesmo código. Um rótulo, por sua vez, só tem significado em conjunto com os comandos que podem desviar para ele. Sem uma análise de concorrência e de fluxo, a refatoração poderia mudar a sincronização, invalidar um salto ou alterar o comportamento do teste.

```
synchronized (this) { banco.abrir(); }
```

A.2.10 Expressão lambda ou referência a método

Ocorre na presença de uma expressão lambda ou de uma referência a método, salvo quando aparecem dentro de uma asserção. O suporte foi excluído para simplificar a análise de escopo e de execução adiada. No trecho, a lambda captura `conta` e seu corpo é executado por `forEach`, não no momento em que a função é declarada. Uma lambda também pode capturar atributos ou variáveis locais cuja disponibilidade muda quando o teste é transferido para outra classe. Sem representar essas capturas e o momento da execução, o Róza poderia mover uma configuração para fora de seu contexto e quebrar o teste.

```
contas.forEach(conta -> conta.ativar());
```

A.2.11 Classe anônima ou classe local

Ocorre na presença de uma classe anônima ou de uma classe local. Essa violação foi adotada para simplificar a implementação, pois esses tipos introduzem campos, métodos e escopos próprios dentro do método de teste. No trecho, o corpo de `run` não executa quando o `Runnable` é criado, mas apenas quando ele é invocado. Além disso, a classe pode capturar variáveis e a instância que a contém. Mover somente a criação ou extrair comandos de seu corpo pode alterar essas capturas e o momento dos efeitos. Por isso, o Róza recusa o método para evitar uma transformação incompleta.

```
new Runnable() {
    public void run() { banco.abrir(); }
};
```

A.2.12 Record local

Ocorre na presença de um record declarado no interior de um método. Essa violação foi adotada pelas mesmas razões que recusam a classe local. O record introduz um tipo, um construtor canônico e acessórios no escopo do método, e seu corpo pode incluir um construtor compacto ou métodos próprios. No trecho, `Par` existe apenas dentro de `cria`. Extrair comandos anteriores ou posteriores à declaração, ou movê-los para um método de configuração implícita, exigiria preservar o ponto em que o tipo passa a existir e as capturas de variáveis locais. Sem essa análise, a refatoração poderia produzir código inválido ou alterar o comportamento do teste.

```
@Test void cria() {
    record Par(int a, int b) {}
    Par par = new Par(1, 2);
    assertEquals(1, par.a());
}
```

A.2.13 Expressão explícita `this` ou `super`

Ocorre na presença de `this` ou `super` explícitos. Essa violação evita que a mudança de classe altere o objeto ou o método ao qual a expressão se refere. No trecho, a chamada a `criaBanco` por meio de `super` depende da hierarquia da classe de origem. Depois da redistribuição, a classe gerada pode ter outra superclasse ou nenhuma, tornando a chamada inválida ou fazendo-a alcançar outra implementação. De modo semelhante, `this` passa a representar uma instância da nova classe. Como preservar essas referências exigiria manter ou reconstruir o contexto original, o Róza recusa o método para não quebrar sua compilação ou seu comportamento.

```
super.criaBanco();
```